

避障实验手册

文档版本 01
发布日期 2024-07-05



版权所有 © 华为云计算技术有限公司 2023。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HUAWEI和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为云计算技术有限公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为云计算技术有限公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

避障

第一章 实验准备

1.1. 实验描述

本实验的目标是使用 MindSpore 对各种障碍物进行识别，如楼梯，狗洞、路障标识物，机器狗会根据不同障碍物做出不同的避障反应；当机器狗识别到楼梯后，机器狗步态会转变成高踏步模式，离开楼梯后会按照规划路线继续以正常步态行走；当机器狗识别到狗洞后，机器狗会从正常姿态转变为匍匐姿态，直到通过狗洞；当机器狗识别到路障标识物后，会在距离路障标识物一定距离后，偏转一定方向继续前进，直到越过路障标识物后，按照规定路线继续前进。MindSpore 是华为开源的新一代全场景 AI 计算框架，用于支持包括机器学习、深度学习在内的 AI 应用开发，提供灵活的编程范式和丰富的算法库。在实验过程中，我们收集大量障碍物数据集，对预训练权重进行模型训练和调优，并最终转换模型格式，将其在昇腾开发板上进行推理。通过海康工业相机和华为昇腾开发板结合，实现机器狗避障功能。

1.2. 实验目的

- 了解实现机器狗避障的原理和方法
- 掌握数据集的标注方法
- 学会如何将模型应用于机器狗上
- 如何使用华为云 MindSpore 进行模型训练

1.3. 实验建议

本实验主要是在 MindSpore 上用 Python 编程，模型转换过程可以在自己的 PC 上进行，程序执行则需要在华为昇腾 Atlas 200I DK 开发板（Linux）上执行。实

验中未注明的执行地方的，一律在开发板上执行。

前置基础课：

- (1) MindSpore
- (2) 超声波雷达
- (3) Python 高级语法

1.4. 实验环境

本次实验使用的硬件包括：PC 机、昇腾 Atlas 200I DK 开发板、摄像头。实验开始前，请按照如下步骤将各硬件连接：

- (1) 使用数据线将开发板连接至 PC 机；
- (2) 将摄像头连接至开发板；
- (3) 给开发版供电，确保其能正常开机；

1.5. 背景知识 MindSpore 介绍

什么是 MindSpore?

MindSpore 是一个全场景深度学习框架，旨在实现易开发、高效执行、全场景覆盖三大目标，其中易开发表现为 API 友好、调试难度低，高效执行包括计算效率、数据预处理效率和分布式训练效率，全场景则指框架同时支持云、边缘以及端侧场景。

MindSpore 的总体架构：

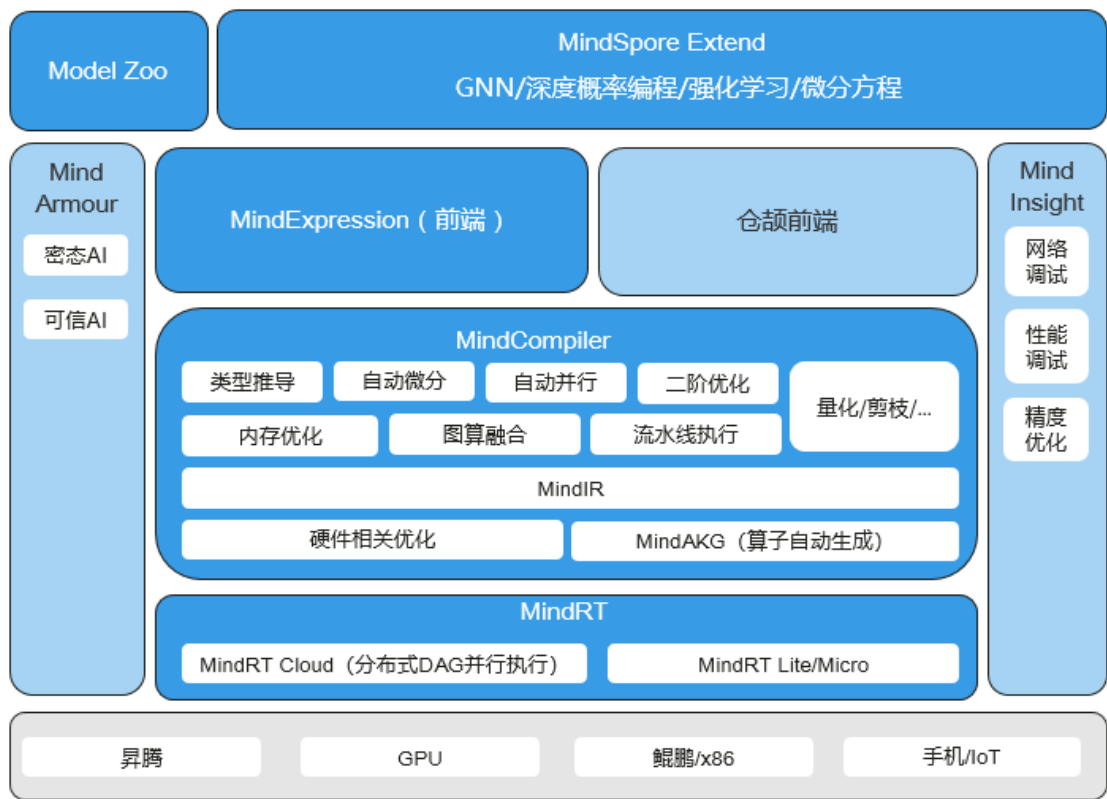
下面介绍主要的扩展层（MindSpore Extend）、前端表达层（MindExpression，ME）、编译优化层（MindCompiler）和全场景运行时（MindRT）四个部分。

MindSpore Extend（扩展层）：MindSpore 的扩展包，期待更多开发者来一起贡献和构建。

MindExpression（表达层）：基于 Python 的前端表达，未来计划陆续提供 C/C++、Java 等不同的前端；MindSpore 也在考虑支持华为自研编程语言前端-仓颉，目前还处于预研阶段；同时也在做与 Julia 等第三方前端的对接工作，引入更多的第三方生态。

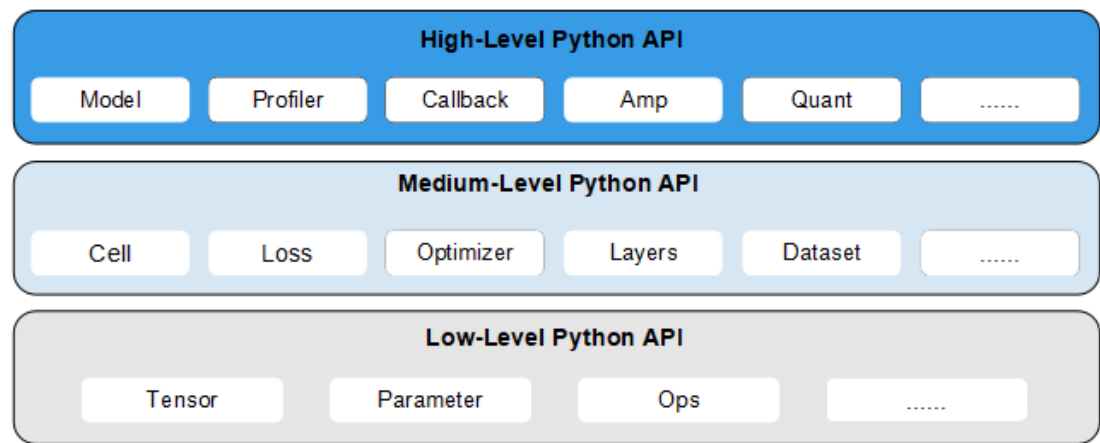
MindCompiler（编译优化层）：图层的核心编译器，主要基于端云统一的 MindIR 实现三大功能，包括硬件无关的优化（类型推导、自动微分、表达式化简等）、硬件相关优化（自动并行、内存优化、图算融合、流水线执行等）、部署推理相关的优化（量化、剪枝等）；其中，MindAKG 是 MindSpore 的自动算子生成编译器，目前还在持续完善中。

MindRT（全场景运行时）：这里含云侧、端侧以及更小的 IoT。



MindSpore 的层次结构：

MindSpore 向用户提供了 3 个不同层次的 API，支撑用户进行网络构建、整图执行、子图执行以及单算子执行，从低到高分别为 Low-Level Python API、Medium-Level Python API 以及 High-Level Python API。



● Low-Level Python API

第一层为低阶 API，主要包括张量定义、基础算子、自动微分等模块，用户可使用低阶 API 轻松实现张量定义和求导计算，例如用户可通过 Tensor 接口自定义张量，使用

ops.composite 模块下的 GradOperation 算子计算函数在指定处的导数。

- Medium-Level Python API

第二层为中阶 API，其封装了低阶 API，提供网络层、优化器、损失函数等模块，用户可通过中阶 API 灵活构建神经网络和控制执行流程，快速实现模型算法逻辑，例如用户可调用 Cell 接口构建神经网络模型和计算逻辑，通过使用 loss 模块和 Optimizer 接口为神经网络模型添加损失函数和优化方式，利用 dataset 模块对数据进行处理以供模型的训练和推导使用。

- High-Level Python API

第三层为高阶 API，其在中阶 API 的基础上又提供了训练推理的管理、混合精度训练、调试调优等高级接口，方便用户控制整网的执行流程和实现神经网络的训练推理及调优，例如用户使用 Model 接口，指定要训练的神经网络模型和相关的训练设置，对神经网络模型进行训练，通过 Profiler 接口调试神经网络性能。

1.6. 实验道具讲解

障碍物：



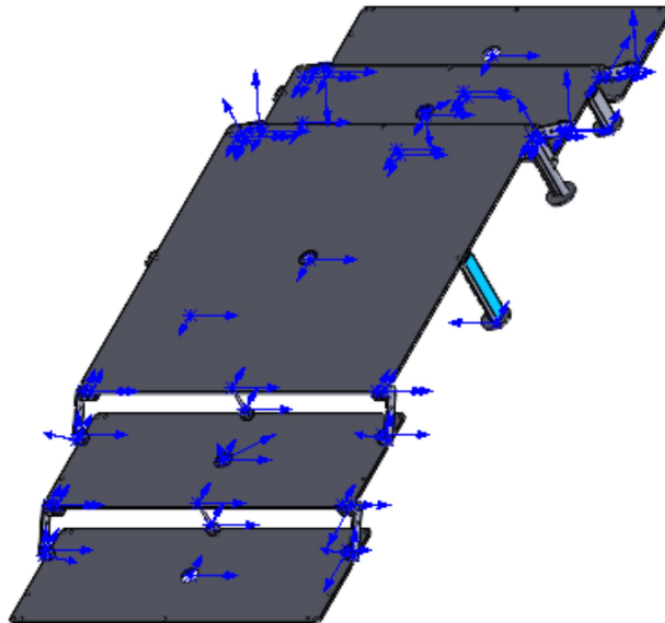
在场景中，通过超声波雷达来检测。

狗洞：

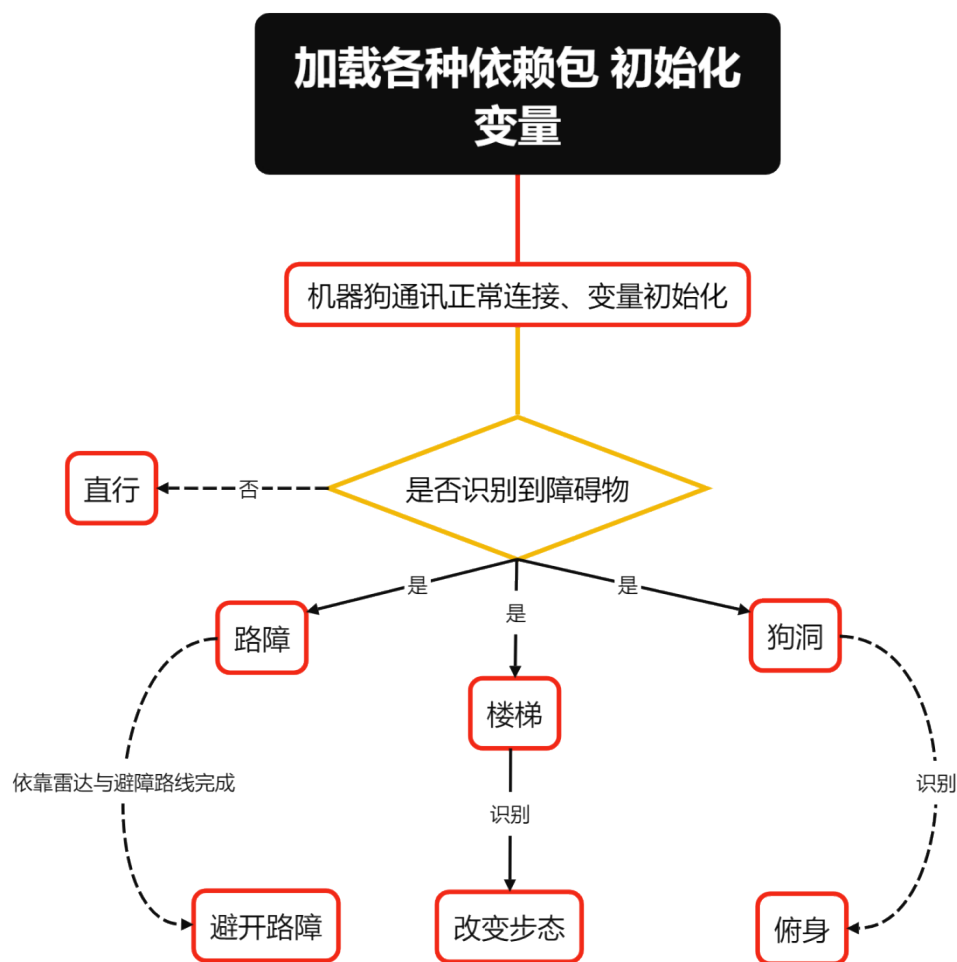


用来模拟需要俯身钻入的障碍物。

楼梯：



第二章 实验过程描述



第三章 环境准备

3.1. 硬件环境准备

(1) SD 卡中系统烧录

向 SD 卡中烧录镜像的步骤如下。

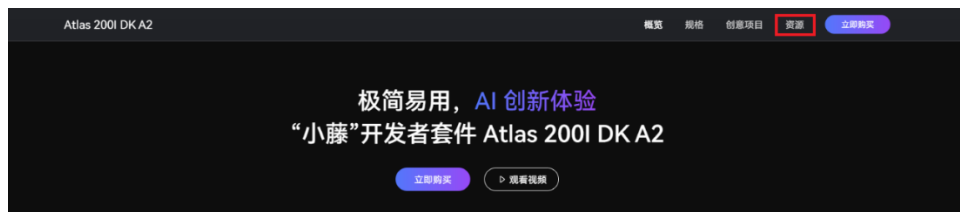
- ① 将 SD 卡插入读卡器，再将读卡器插到 PC 上。



② 在浏览器中进入 [Ascend 官网](#)，选择“产品”->“开发者套件”



③ 点击“资源”，进入资源下载界面



④ 选择“制卡工具下载”，下载完成后进行安装。



⑤ 打开镜像烧录工具 Ascend AI Devkit Imager，选择 Altas 200I DK A2



⑥打开“配置网络信息”，选择“Type-C”选项卡。可以看到 Type-C 接口默认 IP 为 192.168.0.2。选择默认 IP。

镜像网络信息配置

以下为您镜像默认IP信息，您可以根据PC/路由器IP信息重新填写网络信息

网口信息 ETH0 ETH1 Type-C

☐ 自动获取IP地址 ☒ 使用下面的IP地址

IP地址 192 168 0 2

子网掩码 255 255 255 0

默认网关 - - - -

首选DNS服务器 8 8 8 8

备用DNS服务器 114 114 114 114

确定 取消

⑦选择要烧录的 SD 卡

昇腾开发者套件一键制卡工具

1 选择操作

在线制卡 本地制卡 备份SD卡

镜像版本 Desktop-Image_23.0.RC3_ubuntu22.04

Atlas 200I DK A2

配置网络信息

镜像包含OS、NPU驱动固件、CANN、MindX SDK、图形桌面、远程桌面、USB Audio...

OS版本: Ubuntu 22.04 LTS Arm64

固件与驱动版本: 23.0.RC3

CANN版本: 7.0.RC1

发布时间: 2023-11-15

缓存本地磁盘

2 选择SD卡

SD卡大小 62.5 GB

Mass Storage Device USB Device

Mass Storage Device USB Device-62.5 GB

挂载到: F:\H\I\I

Kingston DataTraveler 3.0 USB Device-31.0 GB

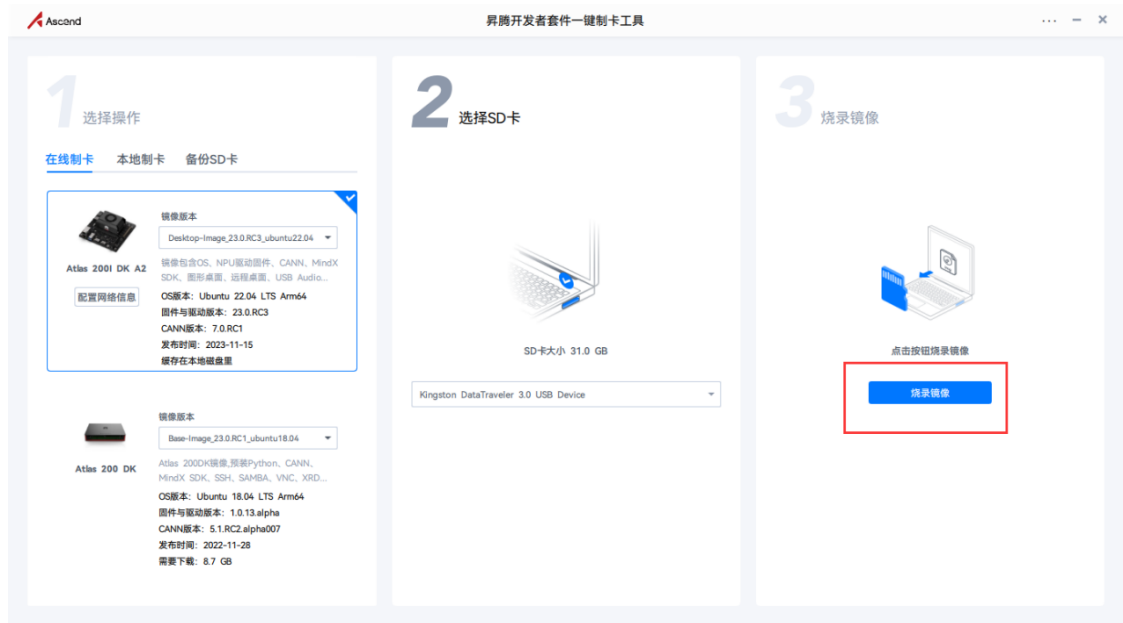
挂载到: G\

3 烧录镜像

点击按钮烧录镜像

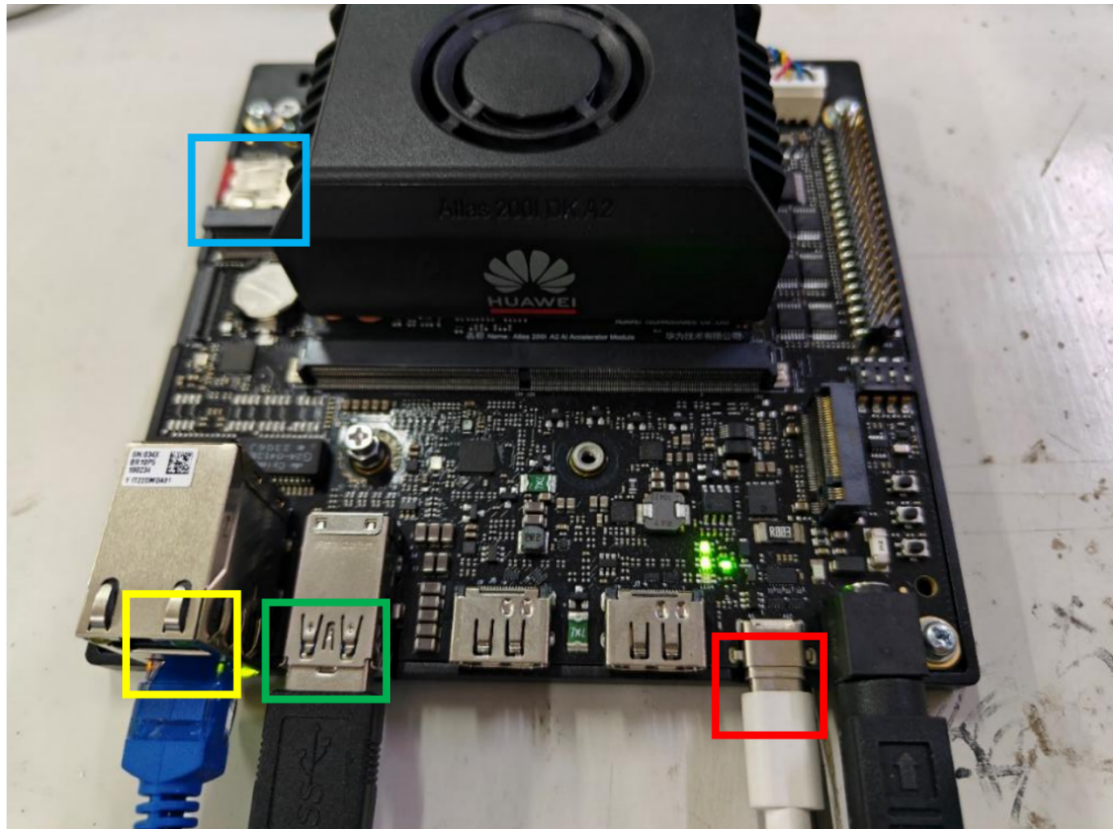
烧录镜像

⑧点击“烧录镜像”，等待镜像烧录完成。



(2) 开发板连接

将烧录好系统的 SD 卡插入到开发板中（下方蓝色框中），昇腾开发板和 PC 机通过 Type-C 线连接，将 Type-C 线和电源线连接到开发板（下方红色框中），并将海康工业相机和机器狗的 USB 线连接到开发板（下方绿色框中）。同时可以连接网线（下方黄色框中），让开发版连接互联网，方便环境依赖包的下载。连接方式如下图所示。



(3) PC 端远程连接开发板

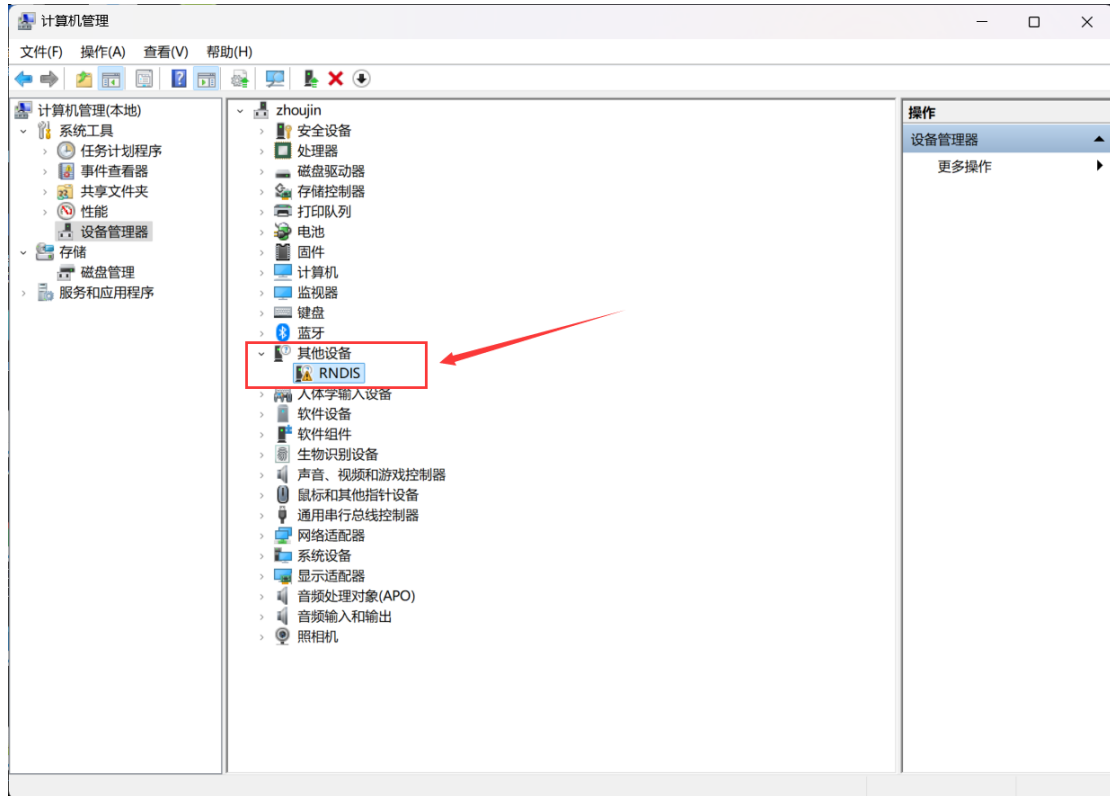
本节介绍当开发者套件通过 Type-C 接口和 PC 连接时,如何在 PC 将接口 IP 地址修改为和开发者套件接口 IP 地址同一个网段（如开发者套件 Type-C 接口为 192.168.0.2，PC 对应 USB 或 Type-C 接口为 192.168.0.101），并使用 SSH 工具远程登录开发者套件。

1) 安装 Windows 的 USB 网卡驱动

本步骤以 Windows 10 系统为例。

①在 PC 上右键单击“此电脑”，选择“更多”→“管理”，进入“计算机管理”界面。

②在“计算机管理”操作界面中选择“设备管理器→“其他设备”，如图所示，RNDIS 为未识别状态。



注意：如果出现 RNDIS，请按以下方法排查：

确保开发者套件 Type-C 接口和 PC 已通过连接线正常连接。

更换具备数据传输能力的 Type-C 数据线（一般手机充电线拥有数据传输能力），继续查看是否出现 RNDIS。

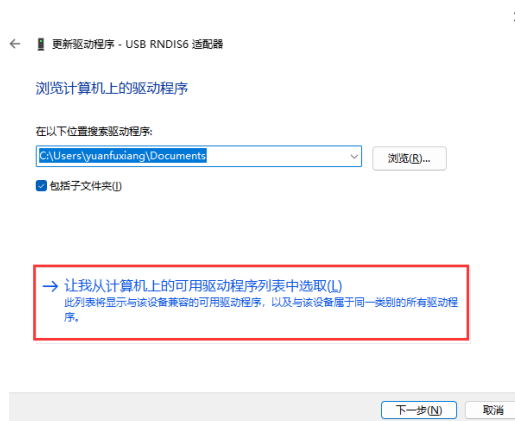
如果尝试现场所有 Type-C 数据线，仍无法出现 RNDIS，则考虑使用 RJ45 网线连接 PC 和开发者套件的以太网口的方式登录开发者套件。

③右键单击“RNDIS”，选择“更新驱动程序(P)”。

④在弹出的“更新驱动程序”→“RNDIS 窗口”中选择“浏览我的计算机以查找驱动程序软件 (R)”。



⑤然后选择“让我从计算机上的可用驱动程序列表中选择(L)”。



⑥在“常见硬件类型”列表中选择“网络适配器”，单击“下一页(N)”。

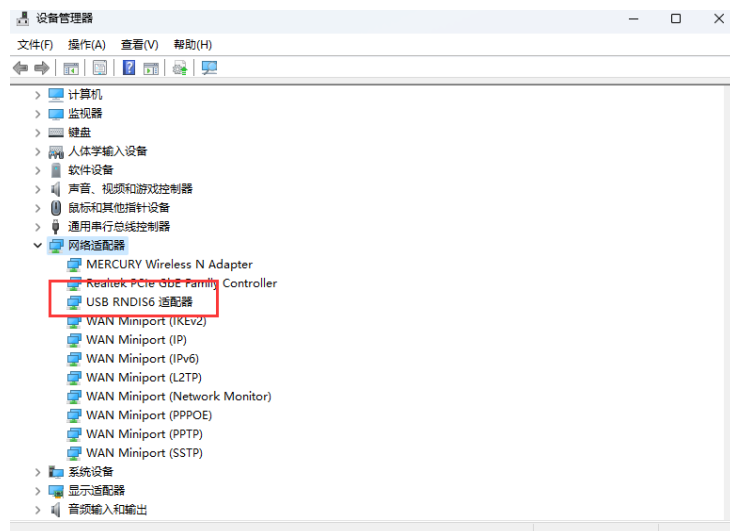


⑦在“选择要为此硬件安装的设备驱动程序”界面中选择“Microsoft”厂商的“USB RNDIS6 适配器”。



⑧单击“下一页”，在弹出的“更新驱动程序警告”窗口选择“是”。

⑨返回“设备管理器”→“网络适配器”，可看到已经正常显示了 USB RNDIS6 适配器的驱动。



(4) 配置 PC 接口 IP 地址

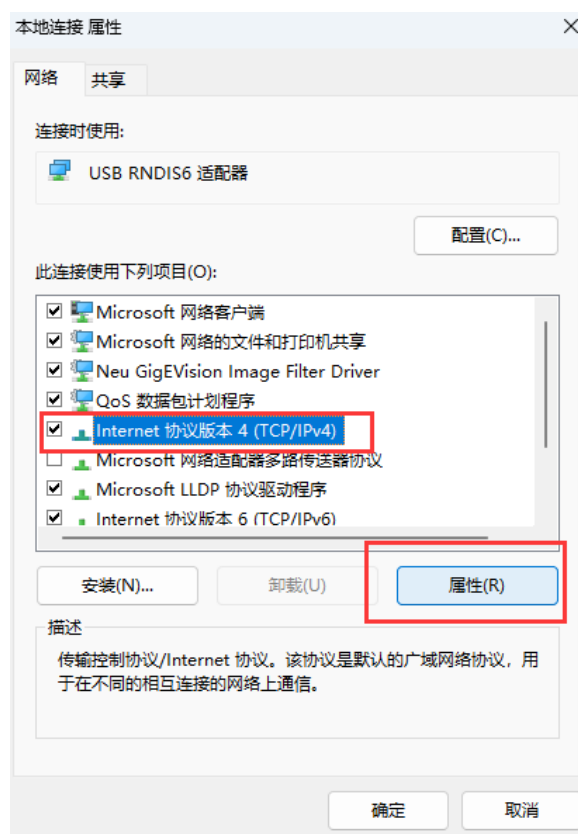
本步骤以 Windows 10 系统为例。

①在 PC 上打开“开始”，单击“设置”按钮，进入“Windows 设置”界面。

②选择“网络和 Internet”，单击“更改适配器选项”。

③鼠标右键单击“本地连接”后鼠标左键单击“属性”进入“本地连接 属性”界面（使用 Type-C 接口连接时一般为“本地连接 x”，x 为数字，以实际显示的数字为准。

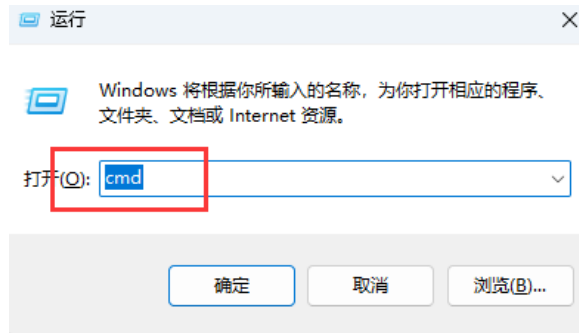
④选择“Internet 协议版本 4 (TCP/IPv4)”，单击“属性”。



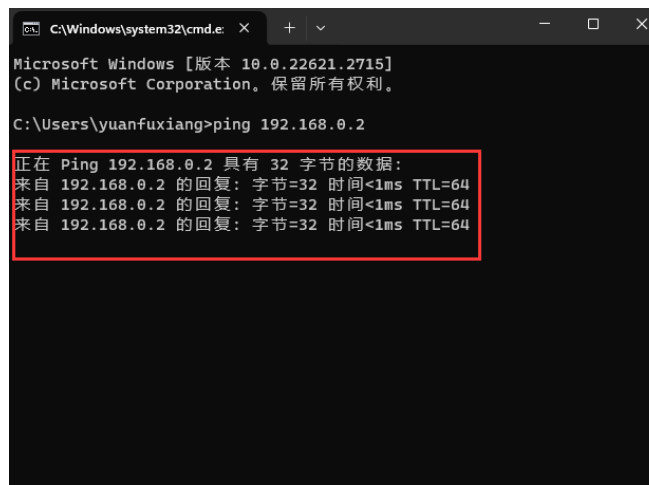
⑤勾选“使用下面的 IP 地址”选项，填写 IP 地址（图示以 192.168.0.101 为例）和子网掩码，默认网关与 DNS 服务器地址为空，单击“确定”保存。



⑥使用快捷键“Win+R”，在运行窗口输入 cmd 进入命令行窗口。



⑦输入 ping 192.168.0.2 命令，查看是否能够访问开发板 IP。

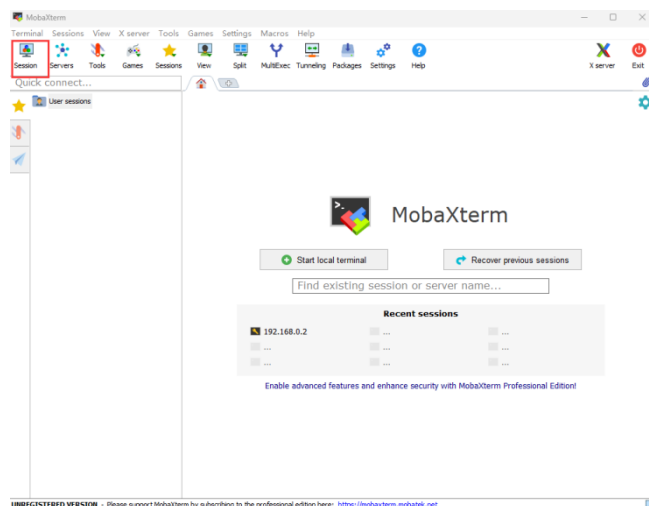


(5) 使用 ssh 远程登录开发板

本文使用 MobaXterm 工具在 PC 上远程登录开发板，单击[下载链接](#)获取 MobaXterm 软件压缩包，解压获得“MobaXterm_Personal_22.2.exe”。

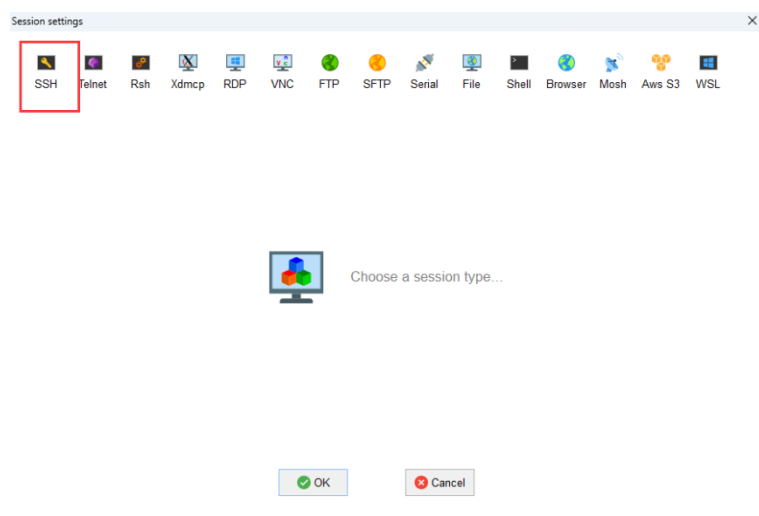
①双击 “MobaXterm_Personal_22.2.exe” 启动 SSH 登录工具。

②单击左上方的 “Session” 进入界面。

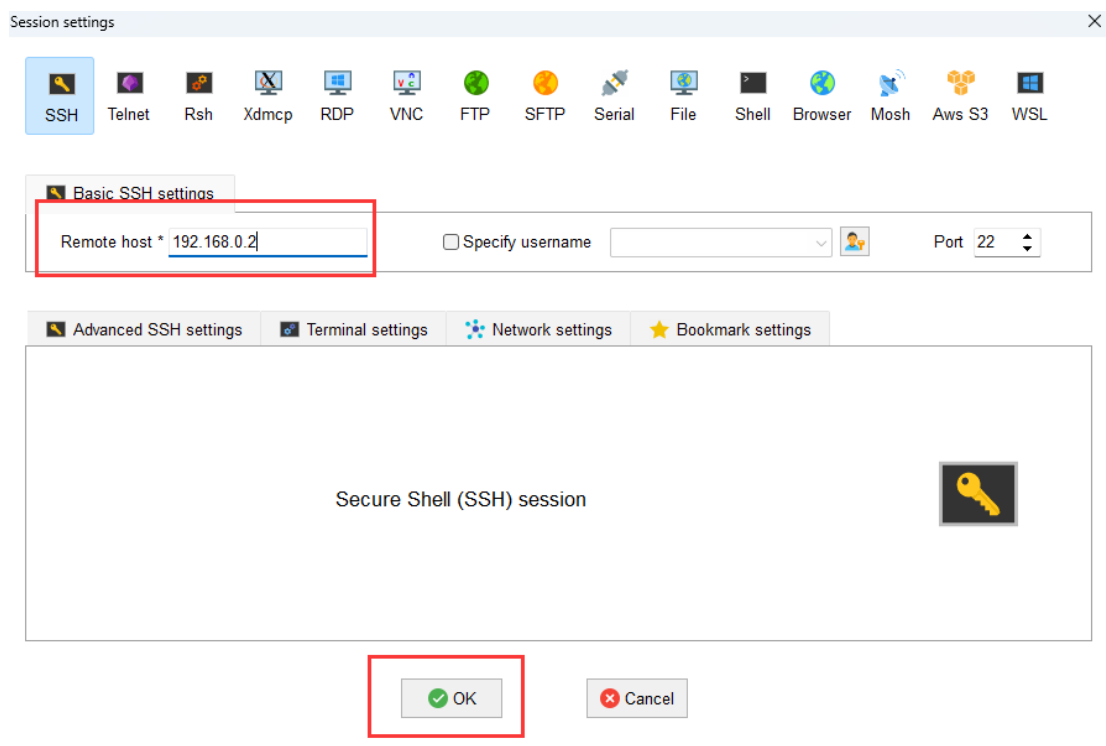


Error! Unknown document property name.

③单击左上方的“SSH”进入 SSH 连接配置界面



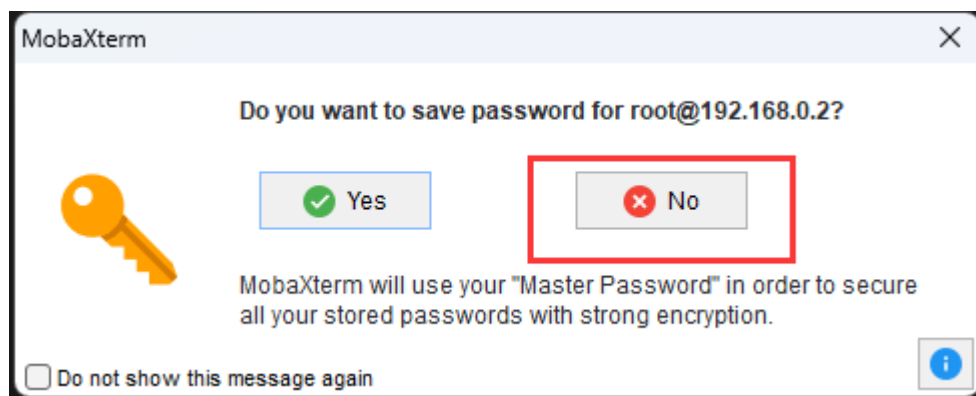
④根据硬件连接方式填写开发者套件连接 PC 的接口实际 IP 地址（一键制卡中配置的 IP 地址）。图示以 192.168.0.2 为例，请根据实际修改。



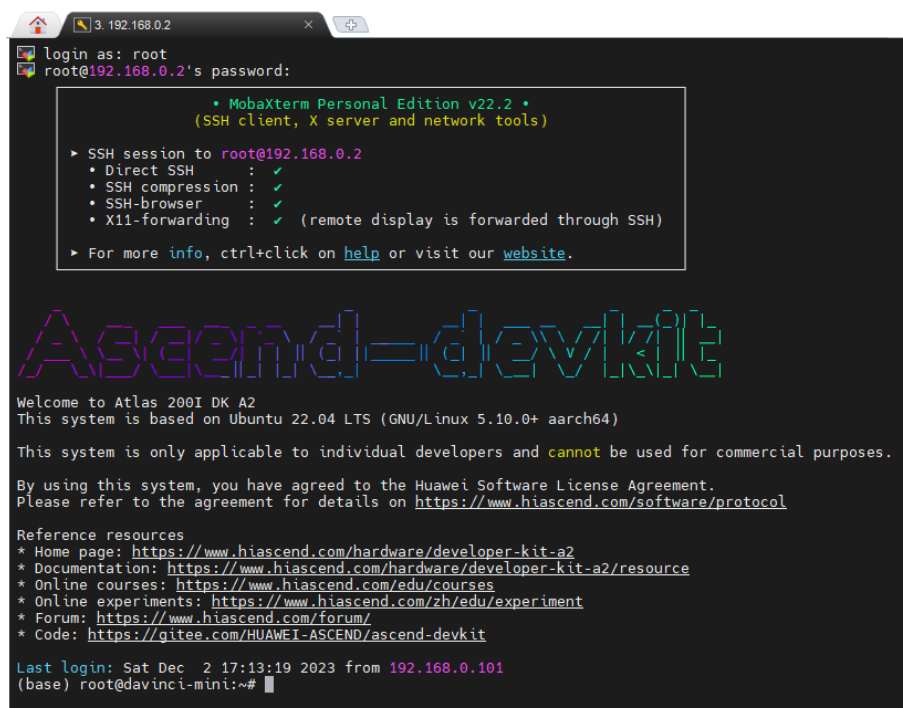
⑤单击“OK”按钮，首次连接开发者套件时，SSH 工具提示是否信任连接的设备，单击“Accept”。



⑥进入远程登录界面后，输入 `root` 用户名登录密码（默认为 `Mind@123`）登录开发者 套件，请修改默认密码，并妥善保管新密码。SSH 工具界面会出现保存密码提示，可以单击“No”，不保存密码直接登录开发者套件。



⑦远程登录开发者套件成功界面如图所示。



4) 机器狗中继连接外网

注: 本教程主要为暂时没有外置无线网卡的用户提供连接外网方案, 教程以中继手机热点为例进行说明, 中继无线路由器同理。

1、电脑连接机器狗的 wifi, 点击电脑右下角的 wifi 图标, 点击属性查看 IPv4 DNS 服务器的 ip 地址。如下图所示。

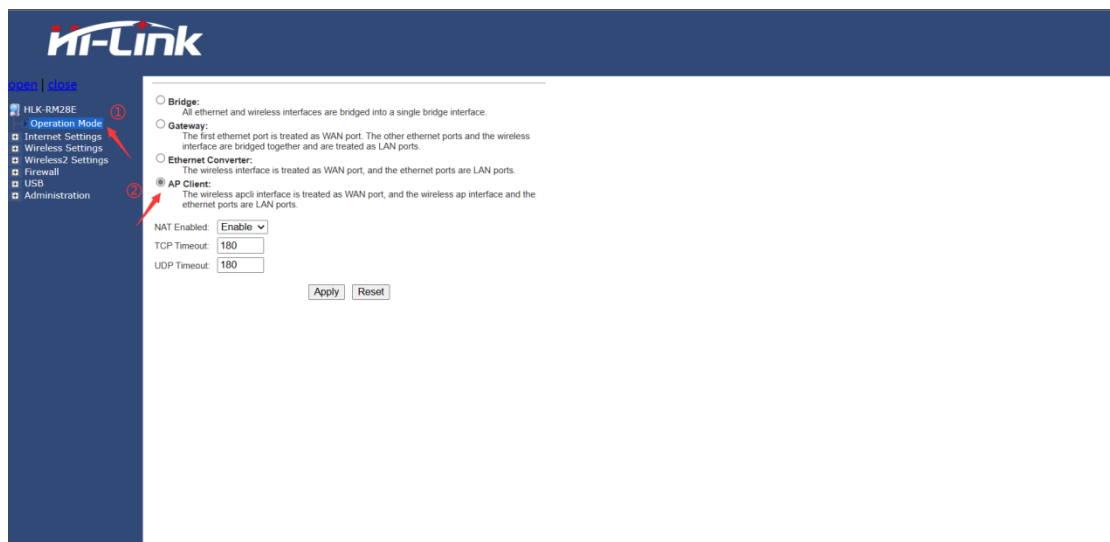


2、打开电脑浏览器, 输入 IPv4 DNS 服务器的 ip, 默认登录用户和密码均为 admin。如下图所示。

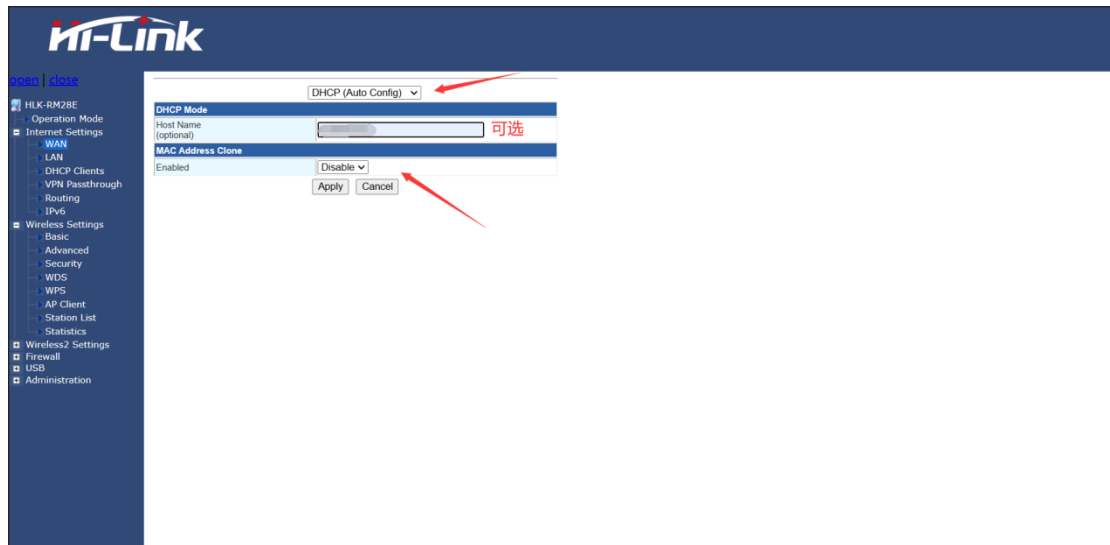


3、打开手机热点。

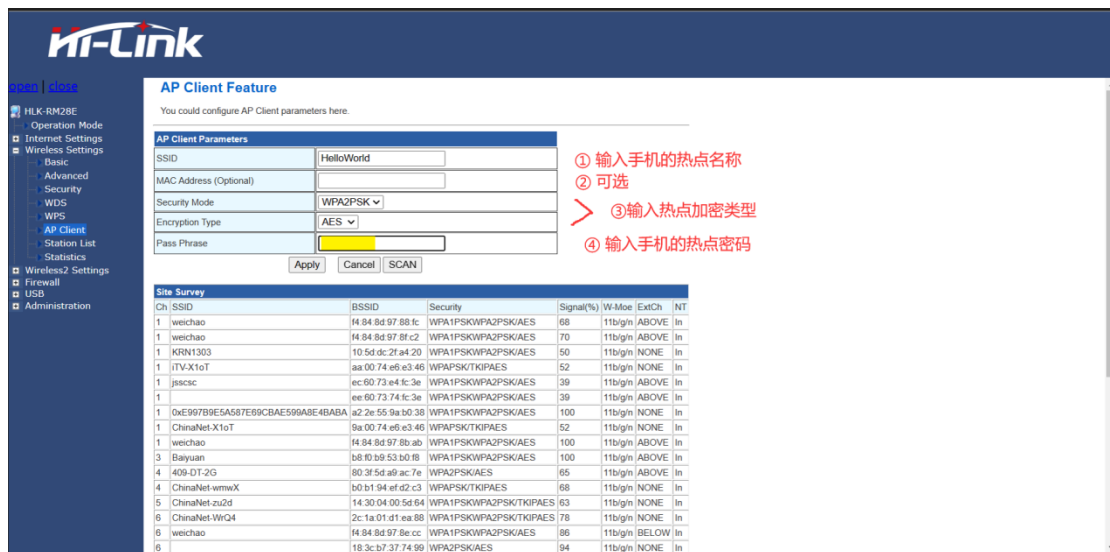
4、进入 Hi-link 设置页面，点击 Operation Mode，选择 AP Client，最后点击 Apply。如下图所示。



5、展开 Internet Settings，点击 WAN，参照下图进行设置，最后点击 Apply。



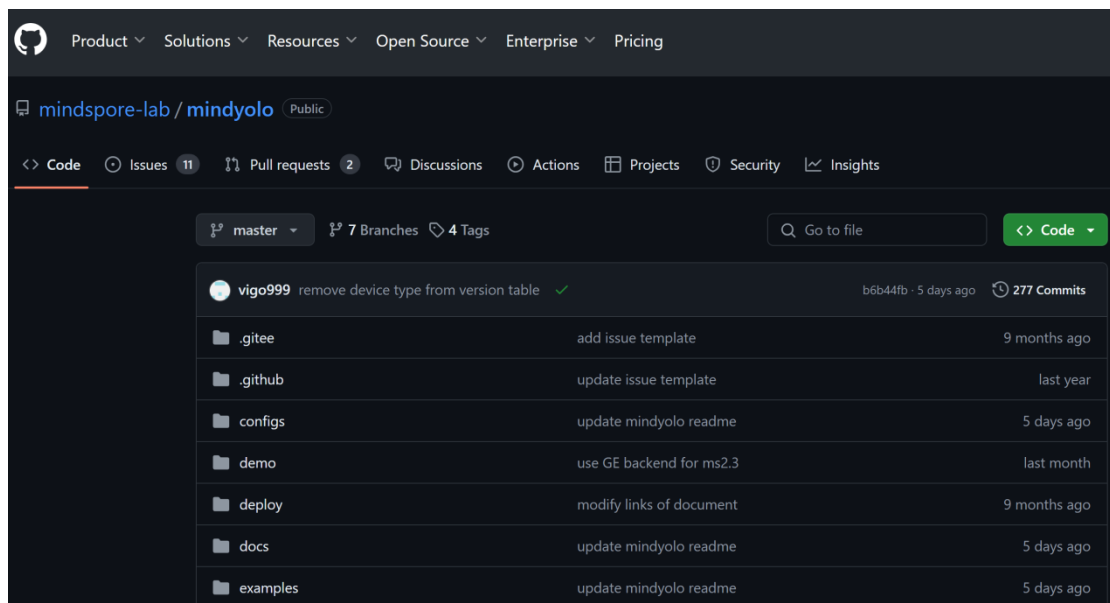
6、展开 Wireless Settings，点击 AP Client，查看 site Survey 表中是否有自己的手机热点名称，如若没有点击 SCAN 进行扫描，在 AP client Parameters 表中参照下图进行设置，最后点击 Apply。



7、重启机器狗，注意中继时热点需一直处于开启状态，电脑连接机器狗的wifi，使用远程软件登录，此时机器狗处于连接外网状态。测试是否有网络，点击浏览器，搜索任意内容，如“百度学术”，能正常显示下图说明正常。



3.2. 下载 mindyolo 项目代码



项目地址: <https://github.com/mindspore-lab/mindyolo.git>

下载项目文件，然后自行保存到一个文件夹中（本实验中保存的路径是 D:\pythonProject\）

3.3. 创建实验环境

创建一个 3.9 版本的环境取名为 mindspore1，指令如下：

Error! Unknown document property name.

```
1. conda create-n mindspore1 python=3.9.2
```

然后进入到 mindspore1 的环境中：

```
1. conda activate mindspore1
```

如果实际效果如下图所示，则证明成功配置了 mindyolo 的实验环境。

```
(base) PS C:\Users\15210> conda activate mindspore1
(mindspore1) PS C:\Users\15210> d:
(mindspore1) PS D:\> cd .\pythonProject\mindyolo\
(mindspore1) PS D:\pythonProject\mindyolo> python --version
Python 3.9.2
```

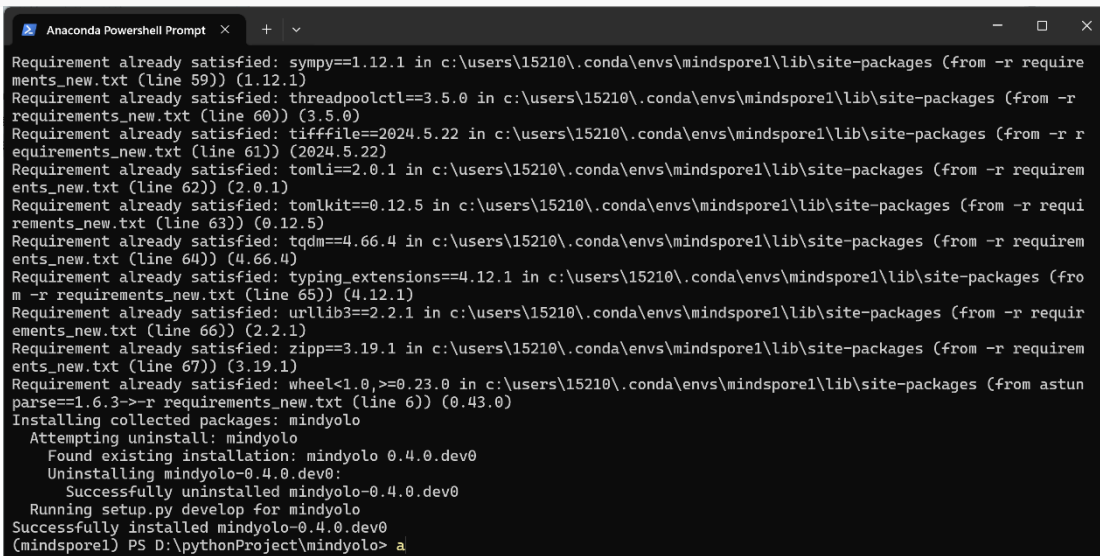
3.4. 安装环境依赖

在这个环境中，我们直接输入指令，安装 requirements_new.txt 这个文件中的环境依赖。

```
1. pip install -r requirements_new.txt
```

因为实验已经实现安装好了，下面的安装就会显示已经安装了，如果下载速度太慢了，可以切换安装源。

```
1. pip install -r requirements_new.txt -
i https://pypi.tuna.tsinghua.edu.cn/simple
```



```
Anaconda Powershell Prompt x + v
Requirement already satisfied: sympy==1.12.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 59)) (1.12.1)
Requirement already satisfied: threadpoolctl==3.5.0 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 60)) (3.5.0)
Requirement already satisfied: tifffile==2024.5.22 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 61)) (2024.5.22)
Requirement already satisfied: tomli==2.0.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 62)) (2.0.1)
Requirement already satisfied: tomliw==0.12.5 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 63)) (0.12.5)
Requirement already satisfied: tqdm==4.66.4 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 64)) (4.66.4)
Requirement already satisfied: typing_extensions==4.12.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 65)) (4.12.1)
Requirement already satisfied: urllib3==2.2.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 66)) (2.2.1)
Requirement already satisfied: zipp==3.19.1 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from -r requirements_new.txt (line 67)) (3.19.1)
Requirement already satisfied: wheel<1.0,>=0.23.0 in c:\users\15210\.conda\envs\mindspore1\lib\site-packages (from astunparse==1.6.3->-r requirements_new.txt (line 6)) (0.43.0)
Installing collected packages: mindyolo
  Attempting uninstall: mindyolo
    Found existing installation: mindyolo 0.4.0.dev0
    Uninstalling mindyolo-0.4.0.dev0:
      Successfully uninstalled mindyolo-0.4.0.dev0
  Running setup.py develop for mindyolo
Successfully installed mindyolo-0.4.0.dev0
(mindspore1) PS D:\pythonProject\mindyolo> a
```

3.5. 开发板环境（机器狗网址配置）

用户可通过 SSH 远程连接到运动主机。

1. 将开发主机连接到机器人 WiFi。
2. 在开发主机上打开 SSH 连接软件，输入 ssh firefly@192.168.1.120，密码为

firefly, 即可远程连接运动主机;

3. 进入 jy_exe, 并打开配置文件:

```
cd /home/firefly/jy_exe/conf/  
vim network.toml
```

4. 配置文件 network.toml 内容如下:

```
ip = '192.168.1.102'  
  
target_port = 43897  
  
local_port = 43893  
  
~
```

5. 修改配置文件第一行中的 IP 地址, 使得代码能够接收到机器狗数据:

- 如果代码在机器人运动主机内运行, IP 设置为运动主机 IP:

```
192.168.1.120;
```

- 如果代码在用户自己的开发主机中运行, 设置为开发主机的静态 IP:

```
192.168.1.xxx;
```

6. 重启运动程序使配置生效。

```
cd /home/firefly/jy_exe  
  
sudo ./stop.sh  
  
sudo ./restart.sh
```

> 此电脑 > DATE (D:) > pythonProject > mindyolo >				
排序 查看 ...				
名称	修改日期	类型	大小	
runs_infer	2024/6/4 14:59	文件夹		
runs_test	2024/6/4 15:57	文件夹		
src	2024/6/28 15:53	文件夹		
tests	2024/6/4 9:06	文件夹		
tutorials	2024/6/4 9:06	文件夹		
.gitignore	2024/6/4 9:06	Git Ignore 源文件	1 KB	
CONTRIBUTING.md	2024/6/4 9:06	Markdown 源文件	8 KB	
data.onnx	2024/6/28 16:51	ONNX 文件	23,699 KB	
GETTING_STARTED.md	2024/6/4 9:06	Markdown 源文件	3 KB	
GETTING_STARTED_CN.md	2024/6/4 9:06	Markdown 源文件	3 KB	
LICENSE.md	2024/6/4 9:06	Markdown 源文件	12 KB	
ma-pre-start.sh	2024/6/4 9:06	SH 源文件	1 KB	
mkdocs.yml	2024/6/4 9:06	Yaml 源文件	5 KB	
MODEL_ZOO.md	2024/6/4 9:06	Markdown 源文件	10 KB	
package.sh	2024/6/4 9:06	SH 源文件	1 KB	
README.md	2024/6/4 9:06	Markdown 源文件	4 KB	
README_CN.md	2024/6/4 9:06	Markdown 源文件	3 KB	
requirements.txt	2024/6/4 9:06	文本文档	1 KB	
requirements_new.txt	2024/6/28 15:45	文本文档	3 KB	
setup.py	2024/6/4 9:06	Python 源文件	3 KB	
test.py	2024/6/4 9:06	Python 源文件	21 KB	
train.py	2024/6/28 15:38	Python 源文件	16 KB	

第四章 工程文件目录结构

4.1. PC

PC 端的文件主要是用来本地训练的项目文件，下面是部分文件的预览。

4.2. Ascend

本章节主要展示的是在开发板上所有的文件，以及每个文件所对应的作用。

本章节主要展示开发板上所有的文件及其对应的作用，以便更好地理解整个项目的结构和功能。

```
(base) root@davinci-mini:/opt/lab/ObstacleAvoidance# tree -I "__pycache__|*.pyc" -L 10
.
├── avoidance_models
│   ├── 1.om
│   ├── fusion_result.json
│   ├── model.onnx
│   └── predefined_classes.txt
├── camera
│   ├── 1.py
│   ├── HKcamera.py
│   └── det_utils.py
├── command
│   ├── __init__.py
│   └── udp_command.py
├── getAndSavePicture.py
├── obstacle_main.py
├── obstacle_model_cap.py
├── robotstatuswatcher
│   ├── __init__.py
│   └── listener.py
├── sendcommand
│   ├── SendToCommand.py
│   ├── __init__.py
│   └── heartbeat.py
├── socketnetwork
│   ├── __init__.py
│   └── network_utils.py
├── speeds
│   ├── __init__.py
│   └── sportsspeed.py
├── threading_utils
│   └── ThreadTemplates.py
```

开发板的文件夹总名为‘ObstacleAvoidance’，如图所示。obstacle_main.py是主程序， getAndSavePicture.py是用来采集数据集的脚本， obstacle_main.py是用来识别障碍物的脚本，该文件夹包含多个子文件夹和文件，每个文件都有其特定的用途。下面是各个文件夹及其主要文件的详细介绍：

Command 文件夹

- **udp_command.py**: 该文件用于存放指令码。它定义了各种控制命令，这些命令通过UDP协议发送到机器狗，以实现对其精确控制。

SendCommand 文件夹

- **heartbeat.py**: 该文件用于发送心跳包。心跳包是一种定期发送的信号，用于检测系统或设备是否在线和正常工作。通过发送心跳包，可以确保机器狗与控制系统之间的通信连接是稳定和可靠的。
- **SendToCommand.py**: 该文件负责发送指令。它包含了一系列函数，用于

将指令发送到机器狗，确保指令能够正确地传达和执行。

SocketNetwork 文件夹

- **network_utils.py**: 该文件是与机器狗建立通信的脚本。它包含了各种网络通信的实用函数，用于初始化和维护与机器狗的通信连接，使得控制指令能够通过网络传输到机器狗。

Speeds 文件夹

- **sportspeed.py**: 该文件定义了机器狗运动的速度方程。通过调整速度参数，可以控制机器狗的运动速度，使其能够按照预定的轨迹和速度执行任务。

ThreadingUtils 文件夹

- **ThreadTemplates.py**: 该文件定义了线程模板。线程是并发执行的基本单位，通过使用线程，可以提高系统的响应速度和处理效率。该文件中的模板可以用于创建和管理多个线程，以实现并行处理。

avoidance_models 文件夹

- 该文件夹主要存放的是用来识别的om模型文件。

在实际开发和调试过程中，了解每个文件的具体作用，有助于快速定位问题并进行相应的修改和优化。

第五章 模型本地化训练及部署

5.1. 配置开发板的 HK 驱动

下载并安装 MVS 驱动文件 (Linux)



海康威视官网下载链接：

<https://www.hikrobotics.com/cn/machinevision/service/download?module=0>

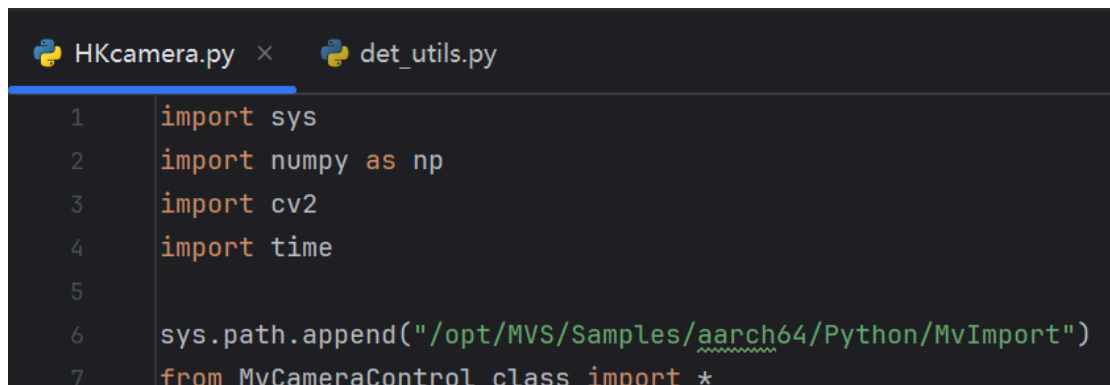


打开 MobaXterm 远程控制，进入开发板安装驱动指令：`dpkg -i MVS-3.0.1_aarch64_20240422.deb`

在代码中使用需要加入包的路径：

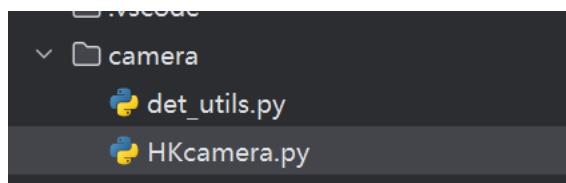
Error! Unknown document property name.

```
sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")from  
MvCameraControl_class import*
```



```
HKcamera.py x det_utils.py  
1 import sys  
2 import numpy as np  
3 import cv2  
4 import time  
5  
6 sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")  
7 from MvCameraControl_class import *
```

使用摄像头的代码：python HKcamera.py



5.2. 编写数据采集脚本

getAndSavePicture.py

```
1. #!/usr/bin/env python  
2. # -*- coding: utf-8 -*-  
3.  
4. import cv2  
5. import time  
6.  
7. import sys  
8. sys.path.append('camera/')  
9. from HKcamera import getImage  
10. from det_utils import get_labels_from_txt, letterbox, scale_coords, nms  
    , draw_bbox # 模型前后处理相关函数  
11.  
12. # 导入定义的机械狗相关的包
```

```

13. from threading_utils.ThreadTemplates import *           # 线程的模板与基
    本轴指令的发送
14. from sendcommand import SendToCommand, heartbeat       # 发送简单指令与
    心跳包
15. from socketnetwork import network_utils               # 通信函数
16. from command.udp_command import *                     # 存放各种结构体
    与状态数据、指令
17. from speeds.sportspeed import *                       # 运动精准控制的
    基础版模型
18. from robotstatuswatcher.listener import *             # 监听机械狗状态
19. from HandDistance.ridge_np import *                   # 测量距离
20. #getImage 从 HKcamera 模块导入，用于获取图像。
    get_labels_from_txt, letterbox, scale_coords, nms, draw_bbox 从
    det_utils 模块导入，这些函数通常用于
21. #目标检测模型的前后处理。导入与机械狗相关的包，如线程模板、发送指令、网络通
    信、命令结构体、运动控制和状态监听等。
22. def save_image(img):
23.     image_name = f'train_images/demo/{time.time()}.jpg'
24.     cv2.imwrite(image_name, img)
25.     print(f"图像已保存为: {image_name}")
26.
27. print("按 's' 保存图像，按 'q' 退出程序。")
28.
29. # 主循环
30. #通过 getImage() 获取图像。
31. #使用 cv2.imshow 显示图像。
32. #使用 cv2.waitKey(1) 检测按键事件，如果按下 's'，则调用 save_image 保存图
    像；如果按下 'q'，则退出循环。
33. while True:
34.     img = getImage() # 获取图像

```

Error! Unknown document property name.

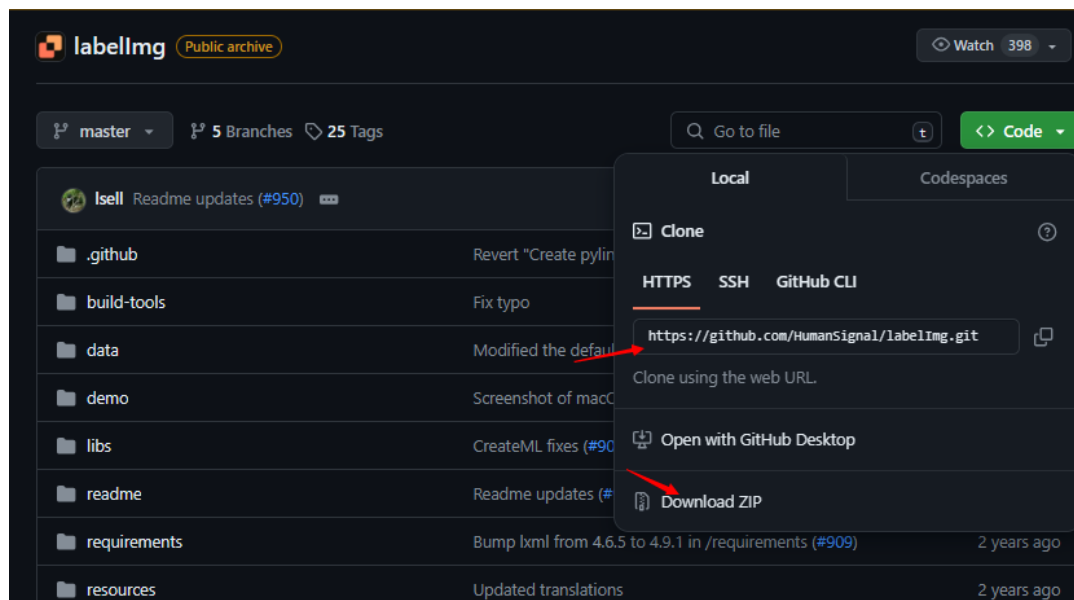
```

35.
36.     cv2.imshow('Camera Feed', img) # 显示图像
37.     key = cv2.waitKey(1) & 0xFF # 检测按键事件
38.
39.     if key == ord('s'):
40.         save_image(img) # 保存图像
41.     elif key == ord('q'):
42.         break # 退出循环
43.
44. # 清理工作
45. cv2.destroyAllWindows()

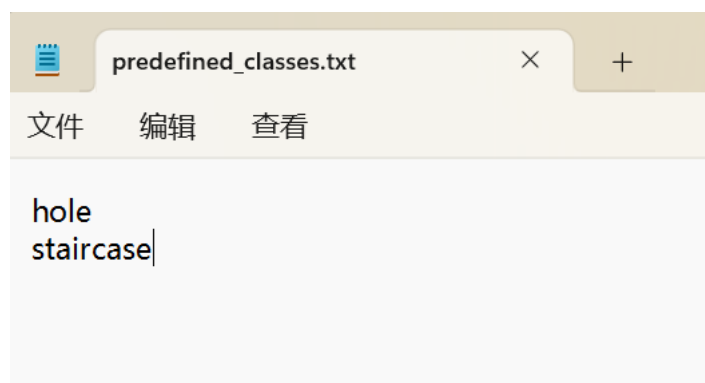
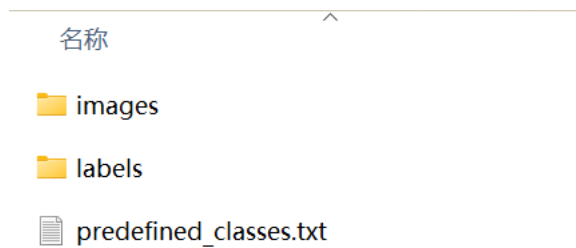
```

5.3. 配置 PC 端数据标注环境

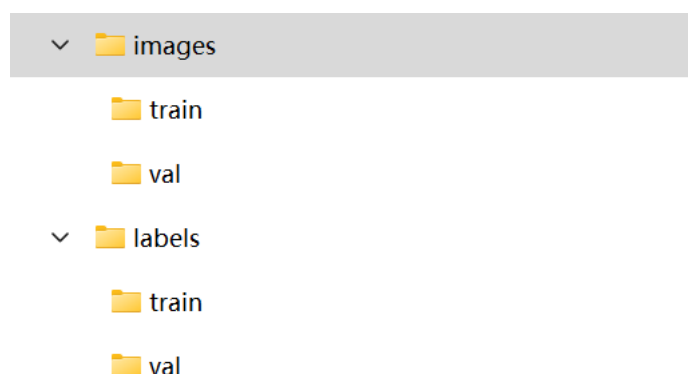
下载数据标注工具 labelimg : <https://github.com/HumanSignal/labelImg>



建立两个文件夹，分别是 `images` 用来存放训练集 `train` 的图片和验证集 `val` 的图片，`labels` 用来存放标注完 `train` 中对应的图片数据标签和 `val` 中的对应的图片数据标签，以及 `predefined_classes.txt` 存放标签类别。



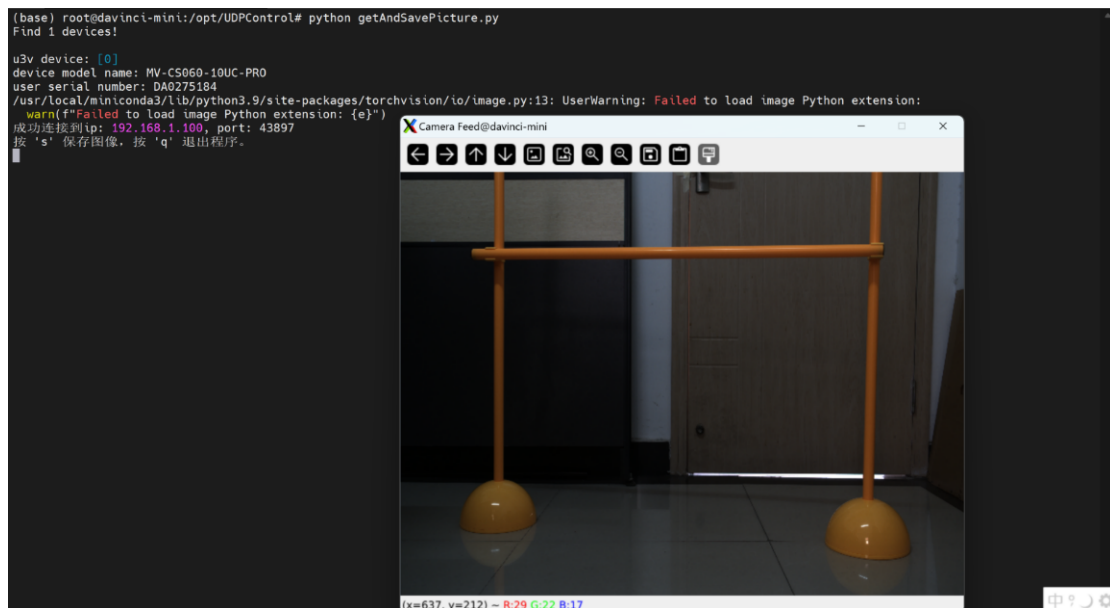
/images/train 对应 labelImg 中的存放目录，而/labels/train 对应 labelImg 的改变存放目录，用于存放标注完图片的数据标签。



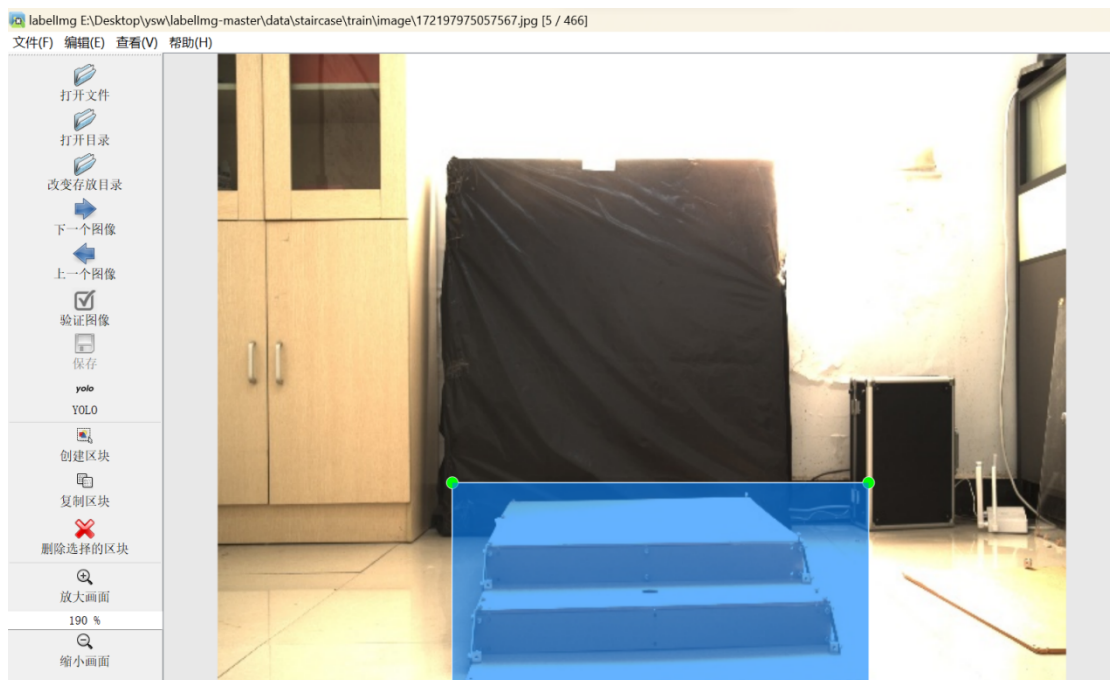
5.4. 标注数据集

5.4.1. 采集数据

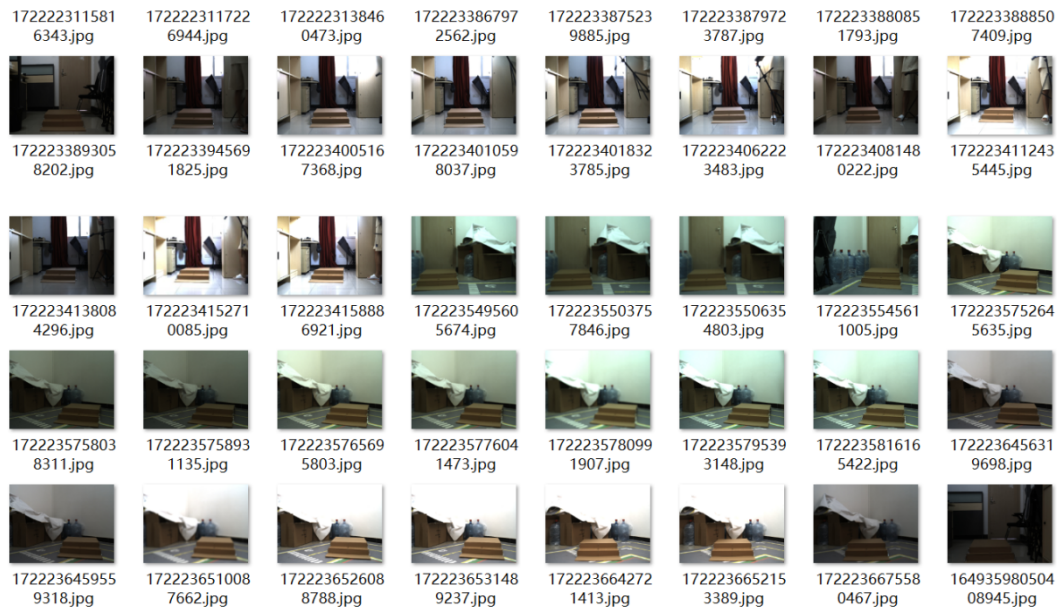
1. 连接开发板，运用脚本采集机器狗视角的图像



2. 进入 pc 存放 labelImg 的目录中, 运行 python labelImg.py

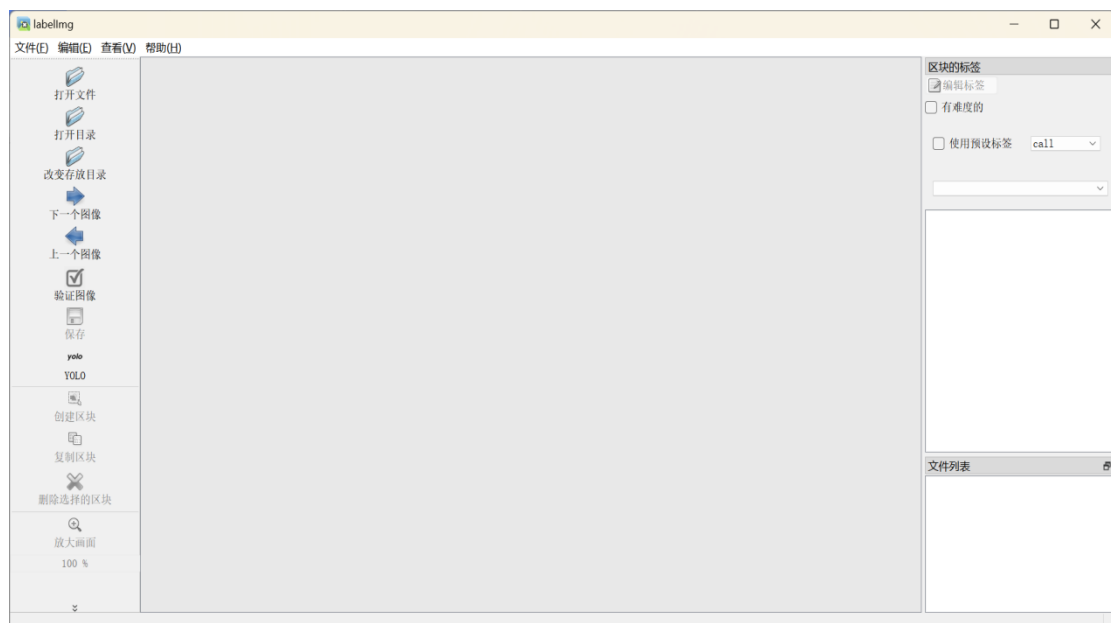


在采集的两类巡检避障产品中, 以其中一类 staircase 为例, 可看到如下图所示。



5.4.2. 使用 labelImg 进行数据集标注

- 1、打开 anaconda 命令行界面
- 2、激活我们已有的环境，如果没有就先创建再激活
- 3、输入 `pip install labelImg` 回车，安装标签库
- 4、输入 `labelImg` 回车，打开标注界面，下图我对各个按钮进行了详细解释



- 5、同时在 `predefined_classes.txt` 中，设置所分类的标签名称；在 `data` 文件夹中分别新建 `images`、`label` 文件夹，同时在 `images` 文件夹中，新建 `train` 文件夹，

里面存放训练集智慧物流产品照片，与之对应在 label 文件夹中，新建 train 文件夹，里面用来存放标注好的数据集信息；同时在 images 文件夹中，再新建 val 文件夹，用来存放验证集物流产品照片，与之对应在 label 文件夹中，新建 val 文件夹，里面用来存放标注好的数据集信息

6、在“打开目录”中打开刚刚的 data/images/train 文件夹，在“改变存放目录”中更换保存路径为 data/label/train 文件夹

5.4.3. 数据标定

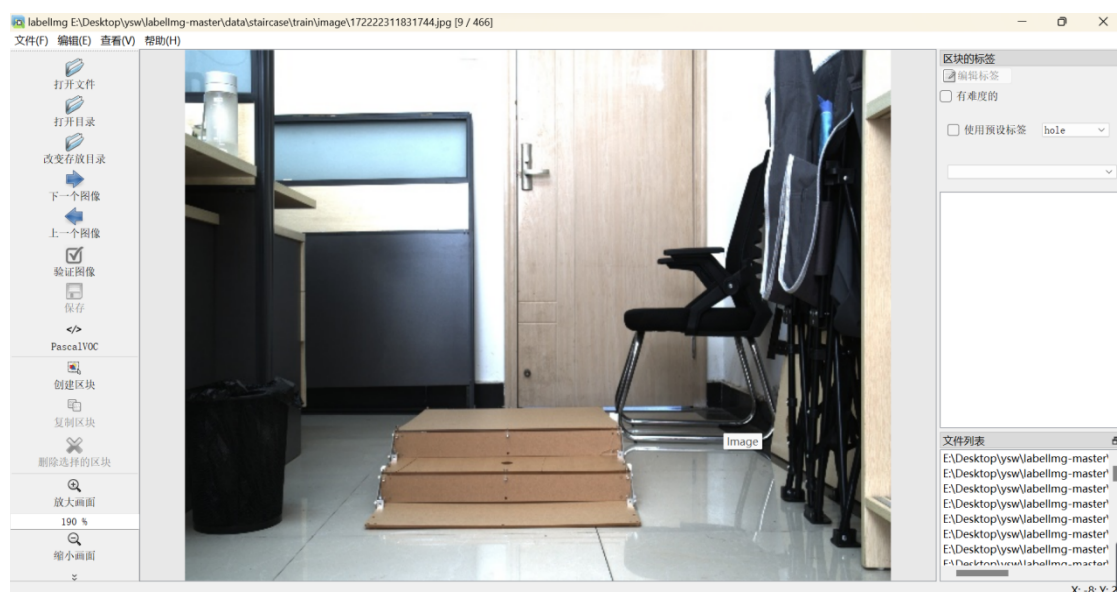
打开 Anaconda Powershell Prompt，输入 pip install labelImg

```
Anaconda Powershell Prompt
(base) PS C:\Users\lly> pip install labelImg
Defaulting to user installation because normal site-packages is not writeable
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/web/simple
Requirement already satisfied: labelImg in c:\users\lly\appdata\roaming\python\python311\site-packages (1.8.6)
Requirement already satisfied: pyqt5 in c:\users\lly\appdata\roaming\python\python311\site-packages (from labelImg) (5.15.9)
Requirement already satisfied: lxml in d:\anaconda\lib\site-packages (from labelImg) (4.9.3)
Requirement already satisfied: PyQt5-sip<13,>=12.11 in d:\anaconda\lib\site-packages (from pyqt5->labelImg) (12.13.0)
Requirement already satisfied: PyQt5-Qt5>=5.15.2 in c:\users\lly\appdata\roaming\python\python311\site-packages (from pyqt5->labelImg) (5.15.2)
```

下载关于 labelImg 的安装包后，进入 labelImg-master 所在文件夹地址

```
(base) PS E:\> cd E:\Desktop\亿生为\labelImg-master
(base) PS E:\Desktop\亿生为\labelImg-master> python labelImg.py
E:\Desktop\亿生为\labelImg-master\labelImg.py:73: DeprecationWarning: sipPyTypeDict() is deprecated, the extension module should use sipPyTypeDictRef() instead
  class MainWindow(QMainWindow, WindowMixin):
[('palm', [(207, 279), (310, 279), (310, 406), (207, 406)], None, None, False)]
```

选择好“打开目录”即图片所在位置、“改变存放目录”即标签存放位置



后输入 `python labelImg.py` 进入数据标注页面，点击”创造区块“，然后出现十字架，选择对应类别：staircase，进行拖动框住标注区域

然后输入标签，点击 staircase，再点击左侧的“保存”标签

在 `data/label/train` 里面可以看到我们的标签，在此介绍 labelimg 的三种标签格式：

- (1) PascalVOC 标签格式，保存为 xml 文件
- (2) YOLO 标签格式，保存为 txt 文件
- (3) CreateML 标签格式，保存为 json 文件

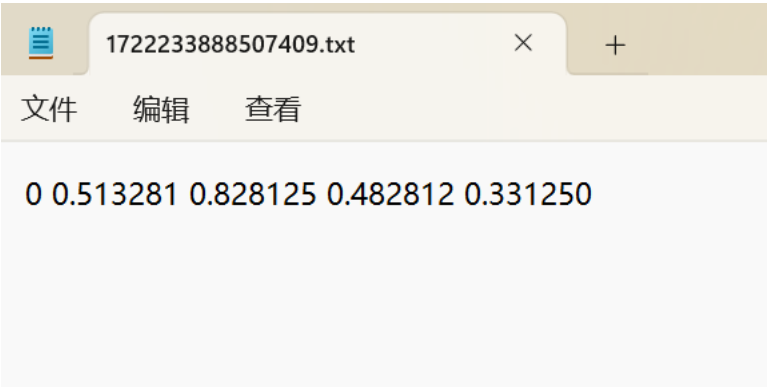
点击“下一个图像”继续标注下一张



5.4.4. 数据预览

数据标注完成后，若标注图片是 `\labeldata\images\train` 中，则可到 `\labeldata\labels\train` 中查看标签文件，`images` 中存放训练集 train 图，`labels` 中存放训练集 train 所对应的数据标签 txt 文件，可看到其文件名对应图片名称，txt 中的四列数据分别对应数据标注区块各个含义，分别对应：类别-classes 框中心坐标，(x_centry_center)，框长宽(width,height)，这里所有的

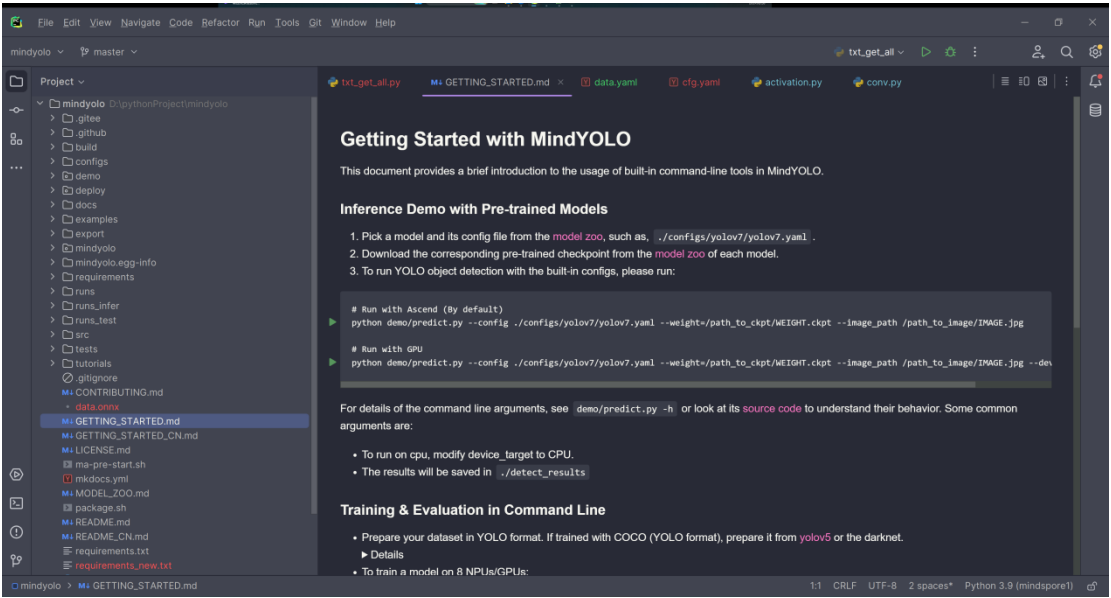
值都进行了归一化具体如下图所示。



```
1722233888507409.txt
文件 编辑 查看
0 0.513281 0.828125 0.482812 0.331250
```

5.5. 启动项目文件

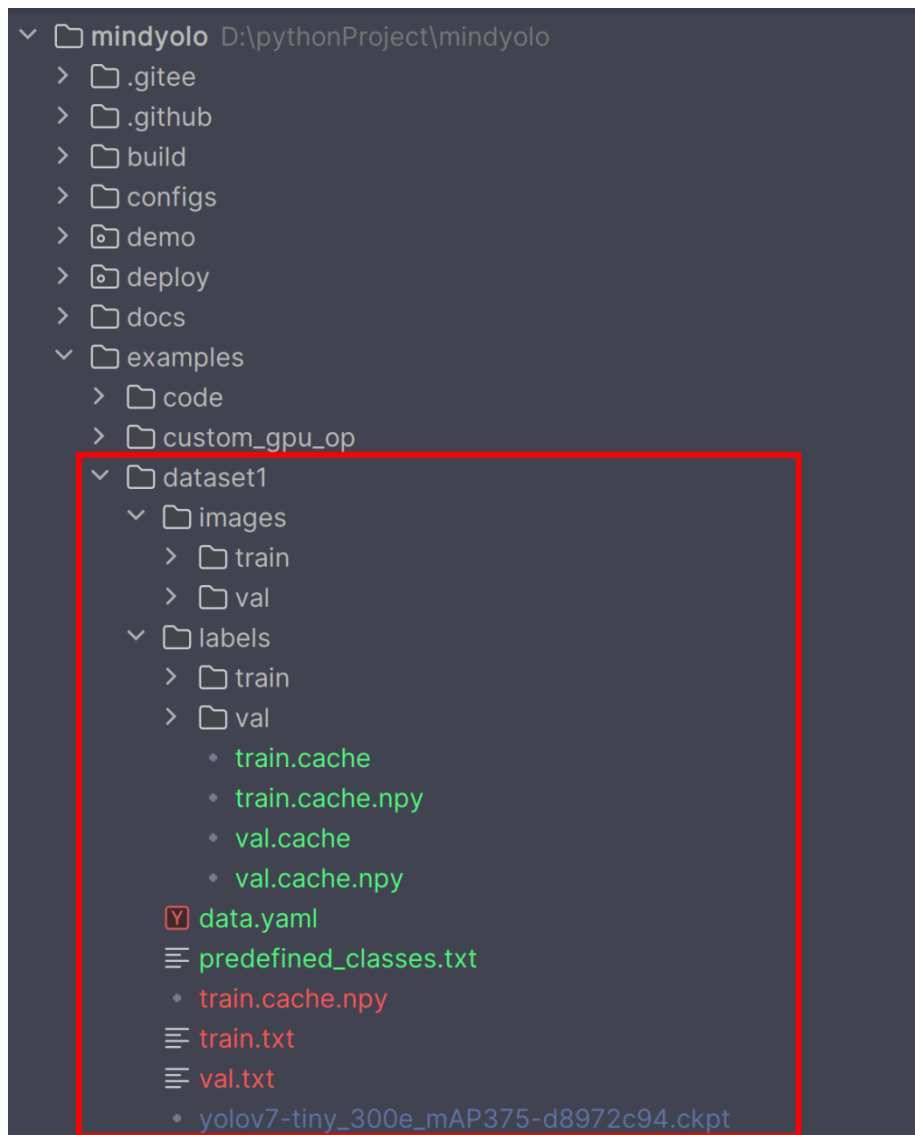
右键项目文件夹，选择使用 `pycharm` 打开。



如果是这样的那么就成功打开了项目文件，同时要注意右下角的 `python` 环境是否正确。

5.6. 数据集格式规范

使用我们事先已经采集好的数据集来训练，下面讲解一下数据集目录的结构：



数据集的存放位置：`examples/dataset1/`

其中 `images/train` 存放的是训练的图片，`images/val` 存放的是验证的图片；`lables` 里面则是存放的标注好的标签 `txt` 文件。`predefines_classes.txt` 里面是物品的种类名称。

`.cache` 和 `.cache.npy` 后缀的文件是训练时候产生的，不需要理会。

`data.yaml` 文件里面的内容如下：

```
1. __BASE__: [  
2.   '../../configs/yolov7/yolov7-tiny.yaml',  
3. ]  
4.  
5. per_batch_size: 1 # 16 * 8 = 128
```

Error! Unknown document property name.

```

6. img_size: 640
7.
8. weight: ./examples/dataset1/yolov7-tiny_300e_mAP375-d8972c94.ckpt
9. strict_load: False
10. conf_thres: 0.6
11. iou_thres: 0.5
12.
13. data:
14.   dataset_name: shwd
15.   train_set: ./examples/dataset1/train.txt
16.   val_set: ./examples/dataset1/val.txt
17.   test_set: ./examples/dataset1/val.txt
18.
19.   nc: 2
20.   names: ['staircase', 'hole' ]
21.
22. optimizer:
23.   lr_init: 0.001

```

配置文件用于配置 YOLOv7 模型训练的参数，在 MindSpore 深度学习框架中的。下面是对各部分配置的解释：

基础配置

```

1. __BASE__: [
2.   '../..//configs/yolov7/yolov7-tiny.yaml',
3. ]

```

这一行表示此配置文件继承自 yolov7-tiny.yaml 文件。__BASE__ 中指定的基础配置文件提供了一些默认设置，而当前配置文件会覆盖或添加一些特定的参数。

训练设置

```

1. per_batch_size: 1

```

Error! Unknown document property name.

2. `img_size: 640`

`per_batch_size: 1`: 每个批次的大小设置为 1，通常在小规模测试或调试时使用较小的批次大小。

`img_size: 640`: 输入图像的尺寸，图像会被调整为 640×640 像素。

预训练模型

1. `weight: ./examples/dataset1/yolov7-tiny_300e_mAP375-d8972c94.ckpt`

2. `strict_load: False`

`weight`: 指定了预训练模型的权重文件路径。

`strict_load: False`: 表示加载预训练权重时不严格匹配所有参数。这对于使用不同结构的预训练模型或部分加载权重非常有用。

检测阈值

1. `conf_thres: 0.6`

2. `iou_thres: 0.5`

`conf_thres: 0.6`: 置信度阈值，只有置信度高于 0.6 的检测结果才会被保留。

`iou_thres: 0.5`: IoU (Intersection over Union) 阈值，用于非极大值抑制 (NMS)，决定哪些框会被保留。

数据集配置

1. `data:`

2. `dataset_name: shwd`

3. `train_set: ./examples/dataset1/train.txt`

4. `val_set: ./examples/dataset1/val.txt`

5. `test_set: ./examples/dataset1/val.txt`

6.

7. `nc: 2`

8. `names: ['staircase', 'hole']`

`dataset_name: shwd`: 数据集名称。

`train_set`、`val_set`、`test_set`: 分别指定训练集、验证集和测试集的文件路径，这些文件通常包含图片的路径列表。

`nc: 2`: 类别数量，这里表示有 2 个类别。

`names: ['staircase', 'hole']`: 类别名称列表，按顺序列出每个类别的名称。

Error! Unknown document property name.

优化器配置

1. yaml
2. optimizer:
3. lr_init: 0.001

lr_init: 0.001: 初始学习率。学习率决定了模型参数更新的步长，较高的学习率会使模型训练更快，但可能会不稳定；较低的学习率训练速度慢但更稳定。

这段配置文件指定了 YOLOv7-tiny 模型训练的具体参数，包括数据集路径、图像大小、批次大小、预训练权重、检测阈值、类别数量及其名称和优化器参数。通过修改这些参数，可以灵活地调整模型训练过程中的各个方面。

接下来解释一下 train.txt 文件和 val.txt 文件还有权重文件 yolov7-tiny_300e_mAP375-d8972c94.ckpt。

首先这个权重文件是自己在官网下载的，也可以在 deploy/README.md 里面找到适合自己的权重模型自己通过点击链接下载。本实验中使用的是 yolov7-tiny_300e_mAP375-d8972c94.ckpt 权重模型。

train.txt 文件和 val.txt 文件里面存放的时候图片路径，加入我们标注完数据之后，没有路径文件，可以使用下面的这个脚本来快速得到。txt_get_all.py 脚本存放在了 examples 文件夹下。代码如下：

```
1. import os
2. sets = ['train', 'val']
3. for j in sets:
4.     # 图片和标签文件的目录
5.     images_dir = fr'D:\pythonProject\mindyolo\examples\dataset1\images\{j}'
6.     labels_dir = fr'D:\pythonProject\mindyolo\examples\dataset1\labels\{j}'
7.     # 要写入的txt 文件路径
8.     output_txt_path = fr'D:\pythonProject\mindyolo\examples\dataset1\{j}.txt'
9.     # 获取所有图片文件的名称
```

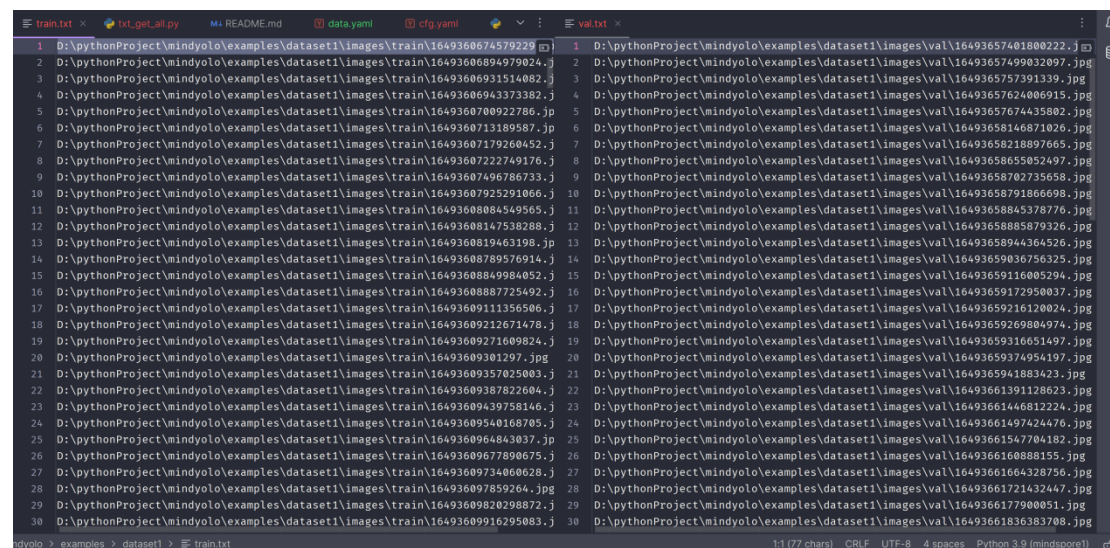


```

10.     image_files = [f for f in os.listdir(images_dir) if f.endswith('.j
    pg'']]
11.     number1, number2 = 0, 0
12.     picture_list = []
13.     file_contents = []
14.     # 打开输出文件
15.     for image_file in image_files:
16.         # 构建图片的完整路径
17.         image_path = os.path.join(images_dir, image_file)
18.         picture_list.append(image_path)
19.         number1 += 1
20.     print(picture_list)
21.     with open(output_txt_path, 'w', encoding='utf-8') as output_file:
22.         for i in range(number1):
23.             print(i)
24.             output_file.write(f"{picture_list[i]}\n")

```

在使用的时候只需要注意 images_dir, labels_dir, output_txt_path 这三个参数的路径即可，运行之后可以得到如下文件：



```

1 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360674579229.jpg
2 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493606894979024.jpg
3 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493606931514082.jpg
4 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493606943373382.jpg
5 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360700922786.jpg
6 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360713189587.jpg
7 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493607179260452.jpg
8 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493607222749176.jpg
9 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493607496786733.jpg
10 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493607925291066.jpg
11 D:\pythonProject\mindyolo\examples\dataset1\images\train\164936080884549565.jpg
12 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493608147538288.jpg
13 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360819463198.jpg
14 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493608789576914.jpg
15 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493608849984052.jpg
16 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493608887725492.jpg
17 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609111356506.jpg
18 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609212671478.jpg
19 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609271609824.jpg
20 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609301297.jpg
21 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609357025003.jpg
22 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609387822604.jpg
23 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609439759146.jpg
24 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609540168705.jpg
25 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360964843037.jpg
26 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609677890675.jpg
27 D:\pythonProject\mindyolo\examples\dataset1\images\train\1649360973406026.jpg
28 D:\pythonProject\mindyolo\examples\dataset1\images\train\164936097859264.jpg
29 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609820298872.jpg
30 D:\pythonProject\mindyolo\examples\dataset1\images\train\16493609916295083.jpg

```

5.7. 训练

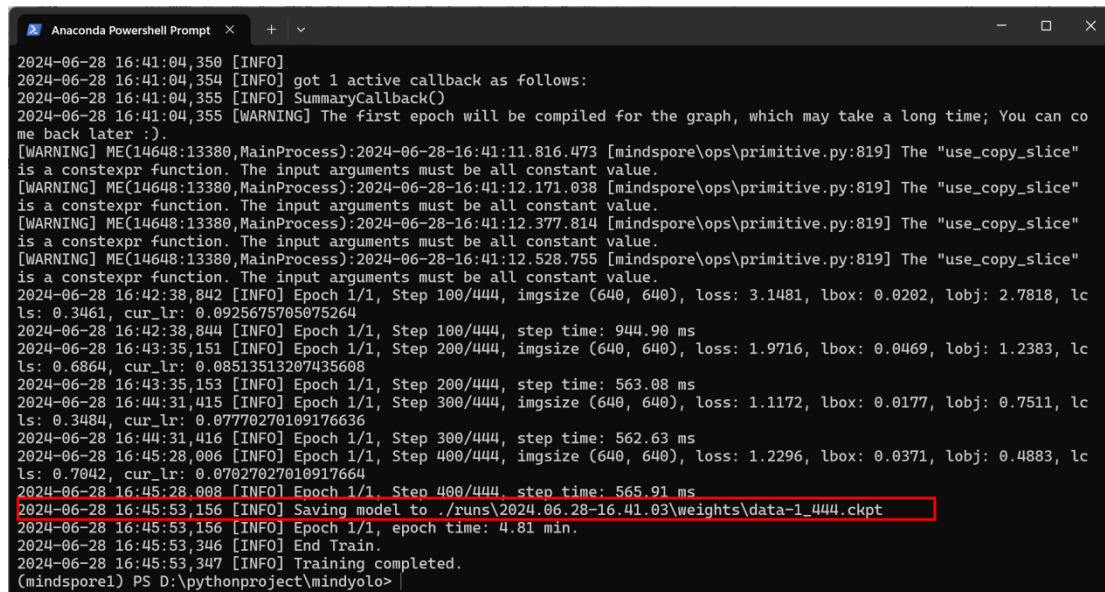
训练的指令如下：

Error! Unknown document property name.


```
1. python train.py --config ./examples/dataset1/data.yaml --  
epochs 300 --device_target CPU
```

这里的 epochs 可以自己定义多少，然后由于框架限制的原因只能使用 cpu 训练，如果需要用 gpu 加速训练，超过了本实验教学的范畴，可以自行线下学习。

输入指令回车之后如果出现了下面的内容，则证明训练成功。（为了做演示，这里值训练 1 轮，使用了 400 多张图片，实际训练需要更多的 epochs 和图片。）

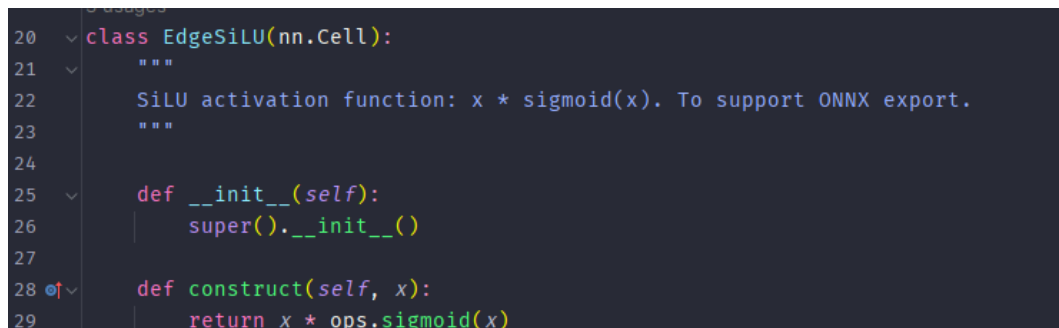


```
2024-06-28 16:41:04,350 [INFO]  
2024-06-28 16:41:04,354 [INFO] got 1 active callback as follows:  
2024-06-28 16:41:04,355 [INFO] SummaryCallback()  
2024-06-28 16:41:04,355 [WARNING] The first epoch will be compiled for the graph, which may take a long time; You can co  
me back later :).  
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:11.816.473 [mindspore\ops\primitive.py:819] The "use_copy_slice"  
is a constexpr function. The input arguments must be all constant value.  
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.171.038 [mindspore\ops\primitive.py:819] The "use_copy_slice"  
is a constexpr function. The input arguments must be all constant value.  
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.377.814 [mindspore\ops\primitive.py:819] The "use_copy_slice"  
is a constexpr function. The input arguments must be all constant value.  
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.528.755 [mindspore\ops\primitive.py:819] The "use_copy_slice"  
is a constexpr function. The input arguments must be all constant value.  
2024-06-28 16:42:38,842 [INFO] Epoch 1/1, Step 100/444, imgsize (640, 640), loss: 3.1481, lbox: 0.0202, lobj: 2.7818, lc  
ls: 0.3461, cur_lr: 0.0925675705075264  
2024-06-28 16:42:38,844 [INFO] Epoch 1/1, Step 100/444, step time: 944.90 ms  
2024-06-28 16:43:35,151 [INFO] Epoch 1/1, Step 200/444, imgsize (640, 640), loss: 1.9716, lbox: 0.0469, lobj: 1.2383, lc  
ls: 0.6864, cur_lr: 0.08513513207435608  
2024-06-28 16:43:35,153 [INFO] Epoch 1/1, Step 200/444, step time: 563.08 ms  
2024-06-28 16:44:31,415 [INFO] Epoch 1/1, Step 300/444, imgsize (640, 640), loss: 1.1172, lbox: 0.0177, lobj: 0.7511, lc  
ls: 0.3484, cur_lr: 0.07770270109176636  
2024-06-28 16:44:31,416 [INFO] Epoch 1/1, Step 300/444, step time: 562.63 ms  
2024-06-28 16:45:28,006 [INFO] Epoch 1/1, Step 400/444, imgsize (640, 640), loss: 1.2296, lbox: 0.0371, lobj: 0.4883, lc  
ls: 0.7042, cur_lr: 0.07027027010917664  
2024-06-28 16:45:28.008 [INFO] Epoch 1/1, Step 400/444, step time: 565.91 ms  
2024-06-28 16:45:53,156 [INFO] Saving model to ./runs/2024.06.28-16.41.03/weights/data-1_444.ckpt  
2024-06-28 16:45:53,156 [INFO] Epoch 1/1, epoch time: 4.81 min.  
2024-06-28 16:45:53,346 [INFO] End Train.  
2024-06-28 16:45:53,347 [INFO] Training completed.  
(mindspore1) PS D:\pythonproject\mindyolo>
```

训练成功之后的模型文件保存了 ./runs/2024.06.28-16.41.03/weights/data-1_444.ckpt 中。

5.8. 配置模型转换算子

将 ckpt 转换为 onnx 模型需要自行添加算子，首先 mindyolo/models/layers 文件夹，然后找到 activation.py 文件在里面添加如下所示代码：



```
20 class EdgeSiLU(nn.Cell):  
21     """  
22     SiLU activation function: x * sigmoid(x). To support ONNX export.  
23     """  
24  
25     def __init__(self):  
26         super().__init__()  
27  
28     def construct(self, x):  
29         return x * ops.sigmoid(x)
```

```
1. class EdgeSiLU(nn.Cell):  
2.     """  
3.     SiLU activation function: x * sigmoid(x). To support ONNX export.
```

```

4.      """
5.
6.      def __init__(self):
7.          super().__init__()
8.
9.      def construct(self, x):
10.          return x * ops.sigmoid(x)

```

然后再找到 conv.py 文件。首先导入我们自己定义的算子

```

1. from .activation import EdgeSiLU # 导入自定义的EdgeSiLU

```

```

6 from .activation import EdgeSiLU # 导入自定义的EdgeSiLU

```

再接着往下滑动，找到这一行

```

activation.py  conv.py x
44 super(ConvNormAct, self).__init__()
45 self.conv = nn.Conv2d(
46     c1, c2, k, s, pad_mode="pad", padding=autopad(k, p, d), group=g, dilation=d, has_bias=False
47 )
48
49 if sync_bn:
50     self.bn = nn.SyncBatchNorm(c2, momentum=momentum, eps=eps)
51 else:
52     self.bn = nn.BatchNorm2d(c2, momentum=momentum, eps=eps)
53 # self.act = nn.SiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)
54 self.act = EdgeSiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)

```

```

1.      # self.act = nn.SiLU() if act is True else (act if isinstance
      e(act, nn.Cell) else Identity)
2.      self.act = EdgeSiLU() if act is True else (act if isinstance
      (act, nn.Cell) else Identity)

```

将原来的 self.act 注释掉，然后换成我们自定义的算子。

```
activation.py x conv.py x
92
93     self.groups = g
94     self.in_channels = c1
95     self.out_channels = c2
96
97     assert k == 3
98     assert autopad(k, p) == 1
99
100     padding_11 = autopad(k, p) - k // 2
101
102     # self.act = nn.SiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)
103     self.act = EdgeSiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)
104
105     if sync_bn:
106         BatchNorm = nn.SyncBatchNorm
```

1. `# self.act = nn.SiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)`
2. `self.act = EdgeSiLU() if act is True else (act if isinstance(act, nn.Cell) else Identity)`

然后大概在 100 多行的位置，也是要修改掉，一共需要修改两处。

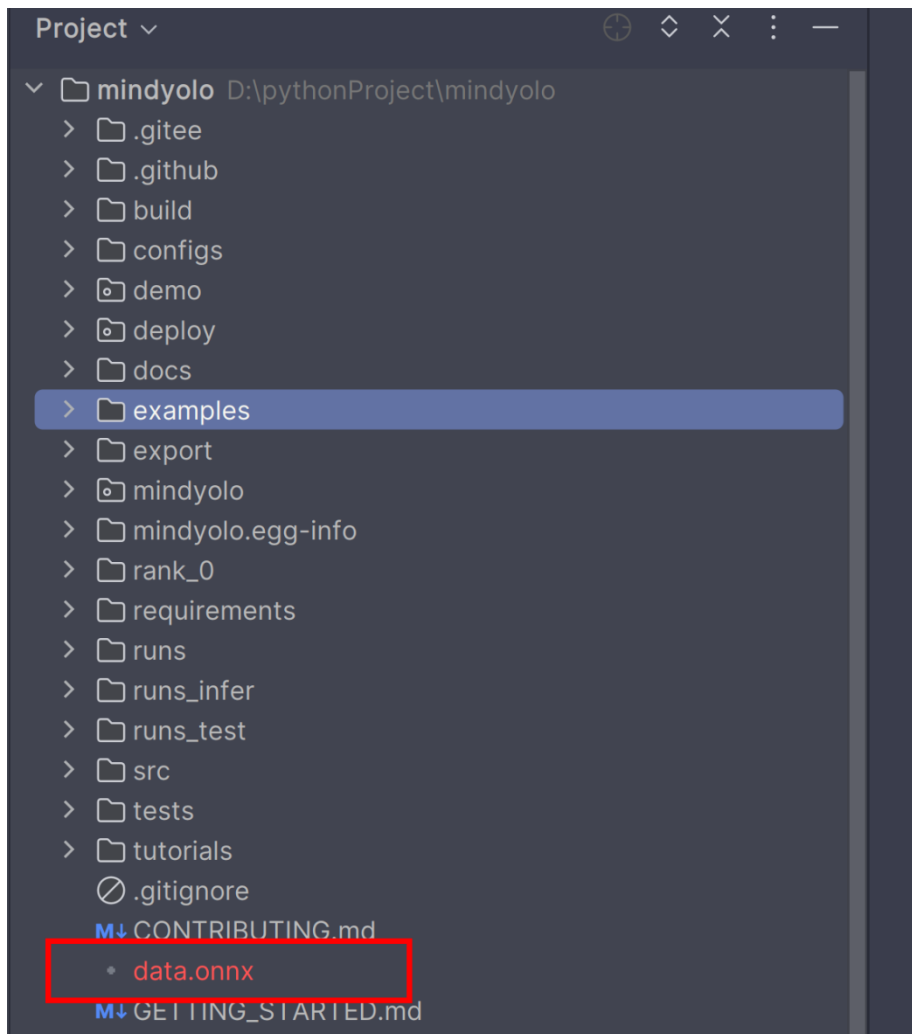
修改之后我们继续输入模型转换的指令：

1. `python ./deploy/export.py --config ./examples/dataset1/data.yaml --weight ./runs\2024.06.28-16.41.03\weights\data-1_444.ckpt --per_batch_size 1 --file_format ONNX --device_target CPU`

转换成功之后的显示如下图

```
Anaconda Powershell Prompt x + v
is a constexpr function. The input arguments must be all constant value.
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.377.814 [mindspore\ops\primitive.py:819] The "use_copy_slice"
is a constexpr function. The input arguments must be all constant value.
[WARNING] ME(14648:13380,MainProcess):2024-06-28-16:41:12.528.755 [mindspore\ops\primitive.py:819] The "use_copy_slice"
is a constexpr function. The input arguments must be all constant value.
2024-06-28 16:42:38,842 [INFO] Epoch 1/1, Step 100/444, imgsize (640, 640), loss: 3.1481, lbox: 0.0202, lobj: 2.7818, lc
ls: 0.3461, cur_lr: 0.0925675705075264
2024-06-28 16:42:38,844 [INFO] Epoch 1/1, Step 100/444, step time: 944.90 ms
2024-06-28 16:43:35,151 [INFO] Epoch 1/1, Step 200/444, imgsize (640, 640), loss: 1.9716, lbox: 0.0469, lobj: 1.2383, lc
ls: 0.6864, cur_lr: 0.08513513207435608
2024-06-28 16:43:35,153 [INFO] Epoch 1/1, Step 200/444, step time: 563.08 ms
2024-06-28 16:44:31,415 [INFO] Epoch 1/1, Step 300/444, imgsize (640, 640), loss: 1.1172, lbox: 0.0177, lobj: 0.7511, lc
ls: 0.3484, cur_lr: 0.07770270109176636
2024-06-28 16:44:31,416 [INFO] Epoch 1/1, Step 300/444, step time: 562.63 ms
2024-06-28 16:45:28,006 [INFO] Epoch 1/1, Step 400/444, imgsize (640, 640), loss: 1.2296, lbox: 0.0371, lobj: 0.4883, lc
ls: 0.7042, cur_lr: 0.07027027010917664
2024-06-28 16:45:28,008 [INFO] Epoch 1/1, Step 400/444, step time: 565.91 ms
2024-06-28 16:45:53,156 [INFO] Saving model to ./runs\2024.06.28-16.41.03\weights\data-1_444.ckpt
2024-06-28 16:45:53,156 [INFO] Epoch 1/1, epoch time: 4.81 min.
2024-06-28 16:45:53,346 [INFO] End Train.
2024-06-28 16:45:53,347 [INFO] Training completed.
(mindspore1) PS D:\pythonproject\mindyolo> python ./deploy/export.py --config ./examples/dataset1/data.yaml --weight ./r
uns\2024.06.28-16.41.03\weights\data-1_444.ckpt --per_batch_size 1 --file_format ONNX --device_target CPU
2024-06-28 16:51:38,629 [WARNING] Parse Model, args: nearest, keep str type
2024-06-28 16:51:38,651 [WARNING] Parse Model, args: nearest, keep str type
2024-06-28 16:51:38,727 [INFO] number of network params, total: 6.037945M, trainable: 6.023106M
2024-06-28 16:51:38,899 [INFO] Load checkpoint from [./runs\2024.06.28-16.41.03\weights\data-1_444.ckpt] success.
2024-06-28 16:51:42,292 [INFO] Export completed.
(mindspore1) PS D:\pythonproject\mindyolo>
```

可以看到上面显示 export completed 说明模型转换成功，转换之后的模型文件保留在了与 train.py 的同级目录下



data.onnx 模型文件，就是我们想要的模型文件

5.9. 开发板转换模型

我们再将 data.onnx 文件上传到开发板子中，运行下面的指令：

```
1. atc --model=data.onnx --framework=5 --soc_version=Ascend310B4 --output=best --input_format=ND
```

然后等待转换模型

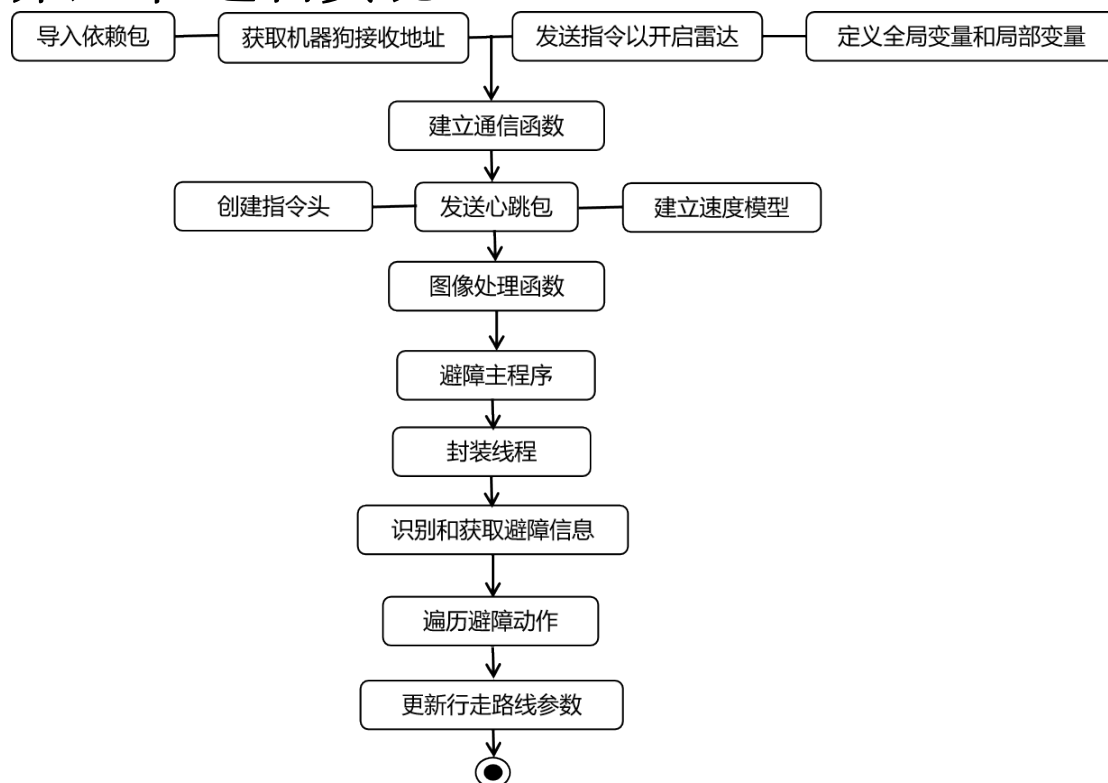
```
(base) root@davinci-mini:/opt/ddd# atc --model=data.onnx --framework=5 --soc_version=Ascend310B4 --output=best --input_format=ND
ATC start working now, please wait for a moment.

(base) root@davinci-mini:/opt/ddd# atc --model=data.onnx --framework=5 --soc_version=Ascend310B4 --output=best --input_format=ND
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.

(base) root@davinci-mini:/opt/ddd# ls
best.om  data.onnx  fusion_result.json
```

当出现 success 的时候说明转换成功了，可以查看到已经有叫 best.om 的模型文件输出了。

第六章 逻辑实现



6.1. 建立通信函数

简单控制第一步：建立通信链接，`socketnetwork/network_utils.py` 文件定义了一个函数 `setup_socket_and_address`，用于创建一个 UDP 套接字，并设置机器狗的 IP 地址和端口号作为目标地址。该通信链接是所有后续控制和数据传输的基础。不仅仅确保了指令传输的可靠性和及时性，还实现对机器狗的远程控制和监控以及为数据传输提供通道，代码如下：

```
1. import socket
2. from typing import *
3.
4. def setup_socket_and_address(dest_ip = '192.168.1.120', port=43893) -> Tuple[socket.socket, Tuple[str, int]]:
5.     # 创建UDP套接字
```

```

6.         sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM
7.     )
8.         # 设置目标地址
9.         target_address = (dest_ip, port)
10.        # print(target_address)
11.
12.        return sfd, target_address

```

这段 Python 代码定义了一个函数 `setup_socket_and_address`，其目的是设置一个 UDP 套接字并返回该套接字及其目标地址。下面是对代码的逐行分析：

`import socket` - 导入 Python 的 `socket` 库，这个库提供了访问底层网络接口的机制。

`from typing import *` - 从 Python 的 `typing` 模块导入所有类型提示，用于在函数和方法中添加类型注解，以提高代码的可读性和健壮性。

`def setup_socket_and_address(dest_ip = '192.168.1.120', port=43893) -> Tuple[socket.socket, Tuple[str, int]]:` - 定义了一个名为 `setup_socket_and_address` 的函数，它接受两个参数: `dest_ip` 和 `port`，这两个参数分别指定了目标 IP 地址和端口号，默认值分别是 `'192.168.1.120'` 和 `43893`。函数的返回类型被注解为 `Tuple[socket.socket, Tuple[str, int]]`，意味着它将返回一个元组，其中包含一个 `socket` 对象和一个包含 IP 地址和端口号的元组。

`sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` - 创建一个新的 UDP 套接字。`socket.AF_INET` 表示使用 IPv4 地址，`socket.SOCK_DGRAM` 表示使用 UDP 协议。

`target_address = (dest_ip, port)` - 将传入的 IP 地址和端口号组合成一个元组，

这个元组将用作套接字的目标地址。

6.2. 发送心跳包

心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。这对于实时控制和数据传输至关重要。确保系统在运行过程中不会因为通信问题而出现故障，提高系统的整体可靠性。发送心跳包代码位于 `sendcommand/heartbeat.py`，代码如下：

```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  import socket
5.  import struct
6.  import time
7.
8.  def send_udp_heartbeat(sfd, target_address, code=0x210400
9.  01, parameters_size=0, type=0, heartbeat_interval=0.25) -> Non
10. e:
11.
12.     # 打包心跳指令数据
13.     heartbeat_command = struct.pack('<III', code, paramet
14.     ers_size, type)
15.
16.     try:
17.         while True:
18.             # 记录发送前的时间
19.             start_time = time.time()
20.
21.             # 发送心跳包
```

```

18.         sfd.sendto(heartbeat_command, target_address)
19.
20.         # 发送后的延时
21.         time.sleep(max(0, heartbeat_interval -
22.             (time.time() - start_time)))
23.     except KeyboardInterrupt:
24.         print("Heartbeat sending stopped by user.")
25.     finally:
26.         # 关闭 socket
27.         sfd.close()

```

`network_utils.setup_socket_and_address()` 函数被调用，用于创建一个 UDP 套接字（或其他类型的网络通信接口）并设置机械狗的 IP 地址和端口号作为目标地址。心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。导入所需的模块：`socket` 用于网络通信，`struct` 用于数据打包，`time` 用于时间相关的操作。

定义了 `send_udp_heartbeat` 函数，它接收以下参数：

`sfd`: 一个 `socket` 对象，用于发送数据。

`target_address`: 目标地址的元组，包含 IP 地址和端口号。

`code`: 心跳指令的代码，默认为 `0x21040001`。

`parameters_size`: 参数大小，默认为 `0`。

`type`: 心跳包的类型，默认为 `0`。

`heartbeat_interval`: 心跳包发送间隔，默认为 `0.25` 秒。

使用 `struct.pack` 将心跳指令的数据打包成字节串。参数 `<III` 表示使用小端

格式打包三个整数。

10-22. 在一个无限循环中，执行以下操作：

记录发送前的时间（第 15 行）。

发送心跳包到目标地址（第 18 行）。

计算发送后需要等待的时间，以确保心跳包的发送间隔符合 `heartbeat_interval`（第 21 行）。

23-24. 使用 `try-except` 结构来捕获 `KeyboardInterrupt` 异常，这通常发生在用户使用 `Ctrl+C` 中断程序时。如果捕获到这个异常，会打印一条消息表明心跳发送已被用户停止。

25-27. 使用 `finally` 块来确保无论是否发生异常，`socket` 都会在函数结束时关闭。

启动心跳线程，代码如下：

```
1.  # 建立通讯
2.  sfd, target_address = network_utils.setup_socket_and_address()
3.
4.  # 定义发送心跳包的线程执行体
5.  # 注意*args 必须以 tuple 形式传入，包括 sfd 和 target_address
6.  heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)
7.  heartbeat_thread.start()
```

`heartbeat_thread.start()` 启动了心跳线程，它将按照设定的时间间隔（这里是 0.25 秒）发送心跳包（`MyRepeatThread` 在后文中会解释）。

6.3. 创建指令头

创建指令头主要为了构建通信协议，提高代码的可读性和可维护性以及支持复杂的指令操作。指令头包含了关键信息，如指令码、参数大小和类型，是通信协议的核心组成部分。通过定义指令头，可以构建和解析复杂的通信协议。

```
1.  action_CommandHead = CommandHead()
```

创建了一个 `CommandHead` 对象，这个对象包含了指令的头部信息，如动作代码、序列号等。

```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  import ctypes
5.  import struct
6.  import inspect
7.
8.  class CommandHead:
9.      def __init__(self, code=0, parameters_size=0, type_=0
10. ):
11.         self.code = code
12.
13.         # 指令码
14.
15.         self.parameters_size = parameters_size
16.
17.         # 指令值
18.
19.         self.type_ = type_
20.
21.         # 指令类型
22.
23.
24. class Command:
25.     kDataSize = 256
26.
```

```

17.         def __init__(self, head=None, data=None):
18.             if head is None:
19.                 head = CommandHead()
20.             if data is None:
21.                 data = [0] * self.kDataSize
22.             self.head = head
23.             self.data = data
24.         class JointStateReceived(CommandHead):
25.             def __init__(self, data):
26.                 """
27.
28.                 1.关节信息的头部信息为 4 字节命令字,4 字节长度,4 字节类
   型,后续为正文数据---(后续的所有都是类似)
29.
30.                 2. 1 字节(b),2 字节无符号短整型(H),4 字节的无符号整型
   (I),接着是 12 个 8 字节的双精度浮点数(12d)
31.
32.                 3.code,parameters_size,type_的是继承于 CommandHead
   头的,这样写是为了方便呈现出,数据都是基于 CommandHead 发送和接收的
33.
   这样可以做到避免所有的复杂数据全部发在相同名字的
   Command 中,而是重新定义一个基于 CommandHead 的结构体
34.
35.                 4.“self.code, ” 逗号是为了去除掉由 struct.unpack
   操作产生的元组问题,可以去去掉逗号直接复制
36.
37.                 """

```

```

38.         self.data = data          # 关节的状态数据, 全部接
受, 再由 code 来决定分配给谁

39.         CommandHead.code, = struct.unpack('<I', self.data
[:4])

40.

41.

42.     class JointAngle(CommandHead):

43.         def __init__(self, joint_state_received):

44.             # 特定于关节角度的数据解析

45.             *self.joint_angles, = struct.unpack('<12d', joint
_state_received.data[12:])

46.

47.     class JointSpeed(CommandHead):

48.         def __init__(self, joint_state_received):

49.             # 特定于关节速度的数据解析

50.             *self.joint_speeds, = struct.unpack('<12d', joint
_state_received.data[12:])

51.

52.     class RobotState(CommandHead):

53.         def __init__(self, data):

54.

55.             CommandHead.code, = struct.unpack('<I', data[:4])

56.             CommandHead.parameters_size, = struct.unpack('<I'
, data[4:8])

57.             CommandHead.type_, = struct.unpack('<I', data[8:1
2])

58.             self.robot_basic_state, = struct.unpack('<I', dat
a[12:16])          # 机器人基本运动状态

```

```

59.         self.robot_gait_state, = struct.unpack('<I', data
[16:20])           # 机器人步态信息
60.         self.robot_motion_state, = struct.unpack('<i', da
ta[176:180])       # 机器人动作状态
61.         self.distance_ahead, = struct.unpack('<d', data[-
16:-8])           # 雷达前方的距离
62.         self.rear_distance, = struct.unpack('<d', data[-
8:])
63.
64.
65.         # self.rpy = struct.unpack('<3d', data[20:44])
                # IMU 角度
66.         # self.rpy_vel = struct.unpack('<3d', data[44:68]
)                # IMU 角速度
67.         # self.xyz_acc = struct.unpack('<3d', data[68:92]
)                # IMU 加速度
68.         # self.pos_world = struct.unpack('<3d', data[92:1
16])             # 机器人在世界坐标系中的位置
69.         # self.vel_world = struct.unpack('<3d', data[116:
140])            # 机器人在世界坐标系中的速度
70.         # self.vel_body = struct.unpack('<3d', data[140:1
64])            # 机器人在体坐标系中的速度
71.         # self.touch_down_and_stair_trot = struct.unpack(
'<I', data[164:168])    # 此功能暂时未激活。此数据仅用于占位
72.         # self.is_charging = struct.unpack('<b', data[168
:169])          # 暂时未开放

```

```
73.         # self.error_state = struct.unpack('<I', data[169
:173])         # 暂时未开放
74.
```

类的继承结构

这些类都继承自 **CommandHead**，它们共享一些基本属性和方法。**CommandHead** 定义了基本信息，而他的子类定义了用于解析命令头部的方法和属性。

JointStateReceived 类

作用：这个类用于接收关节状态的原始数据，并且解析出头部信息。

解析过程：使用 **struct.unpack** 从数据的前 4 个字节解析出命令字（code）。这里使用了小端格式（<）和无符号整型（I）。

设计意图：通过继承 **CommandHead**，**JointStateReceived** 可以利用基类中定义的方法来处理命令头信息。

JointAngle 类

作用：专门用于从关节状态数据中解析出关节的角度。

解析过程：构造函数接收一个 **JointStateReceived** 实例，然后从实例的数据中（跳过前 12 个字节的头部信息）解析出 12 个双精度浮点数，这些数表示关节的角度。

数据结构：关节角度数据紧跟在命令头之后，每个角度占用 8 个字节。

JointSpeed 类

作用：与 **JointAngle** 类似，但用于解析关节的速度。

解析过程：同样接收一个 **JointStateReceived** 实例，并从数据中解析出 12 个双精度浮点数表示关节速度。

RobotState 类

作用：解析机器人的整体状态，包括运动状态、步态信息、动作状态、雷达距离等。

解析过程：

首先解析命令头信息，包括命令码、参数大小和类型。

然后解析机器人的基本运动状态、步态信息、动作状态等，每个状态占用 4 个字节。

数据解析的细节

小端格式: `struct.unpack` 中的 `<` 表示小端格式，这是计算机存储多字节数据时的一种字节顺序，其中最低有效字节存储在地址的最低位。

数据类型: 使用不同的格式字符来指定数据类型，例如 `I` 表示无符号整型（4 字节），`d` 表示双精度浮点数（8 字节）。

星号(`*`): 在 `struct.unpack` 中使用星号是为了接收多个返回值，并将它们解包到多个变量中。

设计哲学

模块化：每个类负责解析特定部分的数据，这有助于代码的组织和维护。

扩展性：通过继承 `CommandHead`，新的状态解析类可以轻松地添加，同时复用头部解析逻辑。

灵活性: 通过注释掉一些解析字段，代码提供了在未来添加新功能的可能性。

使用场景

这些类用于一个机器狗的控制系统中，用于实时监控和解析机器人的状态信息，以便于进行决策和控制。

6.4. 建立速度模型

这段代码提供了一组用于控制设备运动的函数，包括直线行驶、左右平移和左右旋转。

每个运动函数都接受一些参数，如速度档位和运动的距离或时间，并计算出所需的时间和速度。代码中使用了条件逻辑来根据不同的情况选择不同的计算公式。旋转函数中使用了栈跟踪来确定调用上下文，代码是为了适应不同的使用场景而设计的。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. from command.udp_command import *           # 存放各种结
   构体与状态数据、指令
5.
6. def go_straight(long, speedgear=3, times = None, obs_void_distance =
   None):
7.     # 档位速度只是参考值，具体速度按照实际来判断，默认速度是3档
8.     speedgear_ditc = {
9.         1:7000,
10.        2:7500,
11.        3:8000,
12.        4:8700,
13.        5:9000,
14.        6:10500,
15.    }
16.
17.     val = speedgear_ditc[speedgear]
```



```

18.         # 如果传入的是时间, 计算出已经走过的距离, Long 传入 9999 是为在特殊情况
           况才能会有下面的情况
19.         if times != None and obs_void_distance != None and long == 99
           99:
20.             speed_per_second_meter = (-7.237e-
           09) * val ** 2 + 0.0002933 * val - 1.66
21.             # 这里要计算出避障的距离的时间消耗
22.             times_obs_void_distance = obs_void_distance / speed_per_s
           econd_meter
23.
24.             long = abs((times -
           times_obs_void_distance) * speed_per_second_meter)
25.
26.             return long
27.
28.         else:
29.             if long < 0:
30.                 val = -val
31.                 speed_per_second_meter = (-1.5059e-08) * val ** 2 -
           (4.1944e-04) * val - 2.1804
32.                 times = abs(long / speed_per_second_meter)
33.             else:
34.                 speed_per_second_meter = (-7.237e-
           09) * val ** 2 + 0.0002933 * val - 1.66
35.                 times = long / speed_per_second_meter
36.             return [times, val]
37.
38.
39. def translate_left_and_right(long, speedgear=3):
40.     # 档位速度只是参考值, 具体速度按照实际来判断, 默认速度是 3 档

```

```

41.     speedgear_ditc = {
42.         1: 14000,
43.         2: 18000,
44.         3: 21000,
45.         4: 24000,
46.         5: 27000,
47.         6: 34000,
48.     }
49.     val = speedgear_ditc[speedgear]
50.     if long < 0:
51.         val = -val
52.         speed_per_second_meter = -(1.6e-05) * val - 0.2256
53.         times = abs(long / speed_per_second_meter)
54.     else:
55.         speed_per_second_meter = (1.517e-05) * val - 0.1748
56.         times = long / speed_per_second_meter
57.     return [times, val]
58.
59.
60. def revolve_left_and_right(angle):
61.     def find_closest_output(input_value):
62.         # 判断角度是否超过了360度，超过了则需要则算掉
63.         if input_value > 360:
64.             input_value = input_value / (input_value % 360)
65.
66.         # 输出到输入的映射字典
67.         output_to_input_map = {
68.             0: 0,
69.             15: 0.1,
70.             30: 0.2,

```

Error! Unknown document property name.

```

71.          45: 0.7,
72.          60: 0.7,
73.          75: 0.9,
74.          90: 1.3,
75.         105: 1.5,
76.         120: 1.9,
77.         135: 2.2,
78.         150: 2.3,
79.         165: 2.5,
80.         167: 2.8,
81.         185: 2.9,
82.         195: 3.1,
83.         210: 3.4,
84.         225: 3.7,
85.         240: 4,
86.         255: 4.3,
87.         270: 4.5,
88.         285: 4.7,
89.         300: 5,
90.         315: 5.3,
91.         330: 5.6,
92.         345: 5.8,
93.         360: 5.9,
94.         }
95.
96.
97.
98.         # 获取所有输出键并将其转换为列表
99.         keys = list(output_to_input_map.keys())
100.        # 寻找与输入值最接近的输出键

```

Error! Unknown document property name.

```

101.         closest_key = min(keys, key=lambda x: abs(x -
            input_value))
102.         # 返回与最接近键关联的输入值
103.         return output_to_input_map[closest_key]
104.
105.     caller_function = StackTraceParser()
106.     # print(caller_function.get_formatted_stack_info())
107.
108.     # 根据栈的追踪, 如果旋转的调用执行不同的旋转方式
109.     if caller_function.get_stack_info()[0]['file'].split('/')[
        1] == 'hand_distance_main.py':
110.         if 8 < abs(angle) < 20:
111.             if angle < 0:
112.                 val = -10000
113.             else:
114.                 val = 10000
115.             times = 0.1
116.         elif 20 <= abs(angle):
117.             if angle < 0:
118.                 val = -10000
119.             else:
120.                 val = 10000
121.             times = 0.2
122.         else:
123.             times, val = 0, 0
124.
125.     else:
126.         times = find_closest_output(abs(angle)) # 获取时间与角度的
            关系
127.         if angle < 0:

```

```
128.         val = -10000
129.     else:
130.         val = 10000
131.
132.     return [times, val]
133.
134.
135.
136.
```

go_straight(long, speedgear=3, times=None):

这个函数用于控制设备直线行驶。

long: 表示要行驶的距离，单位是米。

speedgear: 表示设备的档位，影响速度，其默认值为3。

times: 表示行驶时间，单位是秒。

函数首先定义了一个字典**speedgear_ditc**，它将档位映射到一个速度值，如果**times**不为None且**long**等于9999，函数会根据时间和速度计算行驶的距离。

如果**long**不是9999，函数会根据距离和速度计算所需的时间。

最终，函数返回行驶所需的时间（或距离）和速度值。

translate_left_and_right(long, speedgear=3):

这个函数用于控制设备左右平移。

参数和**go_straight**函数类似，但速度值是不同的字典**speedgear_ditc**。

根据**long**的正负，选择相应的速度计算公式来计算时间。

返回值是平移所需的时间和速度值。

revolve_left_and_right(angle):

这个函数用于控制设备左右旋转。

angle: 表示要旋转的角度，单位是度。

函数内部定义了一个辅助函数`find_closest_output`，用于找到最接近输入角度的输出值。

函数使用了一个映射字典`output_to_input_map`，将角度映射到一个输出值。

还有一个`get_formatted_stack_info`方法，用于确定是哪个文件调用了旋转函数，以便执行不同的旋转逻辑。

根据调用的文件名和角度的大小，函数会设置不同的时间和速度值。

最终，函数返回旋转所需的时间和速度值。

6.5. 图像处理函数

包含了多个用于图像处理和目标检测的函数。共同组成了一套完整的图像处理和目标检测工具，涵盖了图像预处理、坐标变换、非极大值抑制以及检测结果的可视化等各个方面。从图像预处理到最终的检测结果可视化，提供了一套完整的流程。

```
1.
2. # 导入 time 模块，提供各种与时间相关的函数，例如时间戳、休眠等。
3. import time
4.
5. # 导入 OpenCV 库，一个开源的计算机视觉和机器学习软件库，用于图像和视频处理。
6. import cv2
7.
8. # 导入 NumPy 库，一个用于科学计算的 Python 库，提供多维数组对象、派生对象（如
   掩码数组、矩阵）以及用于快速数组操作的函数。
9. import numpy as np
```

```

10.
11. # 导入 PyTorch 库，一个开源的机器学习库，基于 Torch 库，用于深度学习，提供自动微分、GPU 加速等功能。
12. import torch
13.
14. # 导入 torchvision 库，一个 PyTorch 的扩展包，包含处理图像和视频的实用工具和预训练模型。
15. import torchvision
16.
17. # 定义 letterbox 函数，参数 new_shape 指定目标尺寸，默认为 (640, 640)，color 指定边框颜色，默认为灰色 (114, 114, 114)。
18. # auto 参数决定是否使用最小矩形，scaleFill 参数决定是否拉伸图像填充新尺寸，scaleup 参数决定是否放大图像。
19. def letterbox(img, new_shape=(640, 640), color=(114, 114, 114), auto=False, scaleFill=False, scaleup=True):
20.     # 获取当前图像的尺寸 [height, width]
21.     shape = img.shape[:2]
22.
23.     # 如果 new_shape 是一个整数，则将其转换为一个元组 (new_shape, new_shape)
24.     if isinstance(new_shape, int):
25.         new_shape = (new_shape, new_shape)
26.
27.     # 计算新旧尺寸的比例，取宽度和高度比例的最小值作为缩放比例
28.     r = min(new_shape[0] / shape[0], new_shape[1] / shape[1])
29.
30.     # 如果 scaleup 为 False，则限制 r 的值不超过 1，即不放大图像
31.     if not scaleup:
32.         r = min(r, 1.0)
33.

```

```

34.         # 计算缩放后的图像尺寸
35.         new_unpad = int(round(shape[1] * r)), int(round(shape[0] * r))
36.
37.         # 计算宽度和高度的填充量
38.         dw, dh = new_shape[1] - new_unpad[0], new_shape[0] -
            new_unpad[1] # wh padding
39.
40.         # 如果 auto 为 True, 则使用最小矩形, 填充量应为 64 的倍数
41.         if auto:
42.             dw, dh = np.mod(dw, 64), np.mod(dh, 64) # wh padding
43.
44.         # 如果 scaleFill 为 True, 则不进行缩放, 直接使用新尺寸填充
45.         elif scaleFill:
46.             dw, dh = 0.0, 0.0
47.             new_unpad = (new_shape[1], new_shape[0]) # 使用新尺寸
48.             ratio = new_shape[1] / shape[1], new_shape[0] / shape[0]
            # 重新计算宽高比例
49.
50.         # 将填充量分为上下和左右两部分
51.         dw /= 2
52.         dh /= 2
53.
54.         # 如果需要调整图像尺寸, 则进行缩放
55.         if shape[0] != new_unpad[0]:
56.             img = cv2.resize(img, new_unpad, interpolation=cv2.INTER_
                LINEAR)
57.
58.         # 计算上、下、左、右边界的填充量, 并进行四舍五入
59.         top, bottom = int(round(dh - 0.1)), int(round(dh + 0.1))
60.         left, right = int(round(dw - 0.1)), int(round(dw + 0.1))

```



```

61.
62.     # 使用 cv2.copyMakeBorder 函数添加边框, 颜色为 color 指定的颜色
63.     img = cv2.copyMakeBorder(img, top, bottom, left, right, cv2.B
        ORDER_CONSTANT, value=color)
64.
65.     # 返回调整尺寸并添加边框后的图像, 缩放比例, 以及左右和上下的填充量
66.     return img, ratio, (dw, dh)
67.
68. # 定义 non_max_suppression 函数, 用于在推理结果上执行非极大值抑制
    (NMS), 以排除重叠的检测
69. def non_max_suppression(
70.     prediction, # 预测结果, 是模型输出的张量
71.     conf_thres=0.25, # 置信度阈值, 默认为0.25
72.     iou_thres=0.45, # 交并比 (IoU) 阈值, 默认为0.45
73.     classes=None, # 指定要检测的类别列表, 如果为None, 则检测所有类别
74.     agnostic=False, # 是否进行类别不可知的NMS
75.     multi_label=False, # 是否允许单个实例有多个标签
76.     labels=(), # 可选的标签列表, 用于某些模型的自动标签注入
77.     max_det=300, # 每张图像上最多检测的目标数量
78.     nm=0, # 掩码数量, 当前未使用
79. ):
80.     """在推理结果上执行非极大值抑制 (NMS), 以排除重叠的检测框
81.     返回值:
82.         每张图像的检测结果列表, 格式为 (n,6) 张量 [xyxy, conf, cls]
83.         """
84.
85.     # 如果预测结果是一个列表或元组 (例如在使用 YOLOv5 模型的验证模式时),
        则只选择推理输出
86.     if isinstance(prediction, (list, tuple)):
87.         prediction = prediction[0]

```

```

88.
89.     # 获取预测结果所在的设备 (CPU 或 GPU)
90.     device = prediction.device
91.
92.     # 检查是否在使用 Apple MPS (Metal Performance Shaders), 目前 MPS
    对 NMS 的支持不完整
93.     mps = 'mps' in device.type
94.     # 如果正在使用 MPS, 则在执行 NMS 前将张量转换到 CPU
95.     if mps:
96.         prediction = prediction.cpu()
97.
98.     # 获取批量中的图像数量
99.     bs = prediction.shape[0]
100.
101.     # 计算类别数量, 减去 5 是为了去掉模型输出中的 5 个固定元素, nm 是掩码
    数量, 当前未使用
102.     nc = prediction.shape[2] - nm - 5
103.
104.     # 根据置信度阈值过滤预测结果, xc 为置信度大于 conf_thres 的候选预测
    结果
105.     xc = prediction[..., 4] > conf_thres
106.
107.     # 下面的代码将继续实现 NMS 算法, 筛选出最佳的检测结果
108. # 检查置信度阈值是否在 0 到 1 之间
109. assert 0 <= conf_thres <= 1, f'Invalid Confidence threshold {conf
    _thres}, valid values are between 0.0 and 1.0'
110. # 检查交并比阈值是否在 0 到 1 之间
111. assert 0 <= iou_thres <= 1, f'Invalid IoU {iou_thres}, valid valu
    es are between 0.0 and 1.0'
112.

```

```

113. # 设置
114. # min_wh = 2 # 检测框的最小宽度和高度（像素）
115. max_wh = 7680 # 检测框的最大宽度和高度（像素）
116. max_nms = 30000 # 传入 torchvision.ops.nms() 的最大检测框数量
117. time_limit = 0.5 + 0.05 * bs # 执行 NMS 后退出的时间限制（秒）
118. # 如果类别数大于1，则允许每个检测框有多个标签
119. multi_label &= nc > 1
120.
121. # 记录当前时间，用于监控 NMS 的执行时间
122. t = time.time()
123. # 掩码的起始索引，5 是因为模型输出的前5 个值不是类别概率
124. mi = 5 + nc
125. # 初始化输出列表，用于存储每张图像的检测结果
126. output = [torch.zeros((0, 6 + nm), device=prediction.device)] * b
    s
127.
128. # 遍历批量中的每张图像
129. for xi, x in enumerate(prediction):
130.     # 如果没有检测框剩余，则处理下一张图像
131.     if not x.shape[0]:
132.         continue
133.
134.     # 如果启用自动标注，则将先验标签添加到预测结果中
135.     if labels and len(labels[xi]):
136.         lb = labels[xi]
137.         # 创建一个零张量用于存放自动标注的标签信息
138.         v = torch.zeros((len(lb), nc + nm + 5), device=x.device)
139.         # 将标签的边界框信息复制到新张量
140.         v[:, :4] = lb[:, 1:5]
141.         # 设置置信度为1.0

```

```

142.         v[:, 4] = 1.0
143.         # 设置类别标签
144.         v[range(len(lb)), lb[:, 0].long() + 5] = 1.0
145.         # 将自动标注的标签信息添加到预测结果中
146.         x = torch.cat((x, v), 0)
147.
148.         # 计算置信度，将对象置信度与类别置信度相乘
149.         x[:, 5:] *= x[:, 4:5]
150.
151.         # 将中心点坐标和宽高转换为左上角和右下角坐标
152.         box = xywh2xyxy(x[:, :4])
153.         # 获取掩码信息，如果没有掩码，则为零列
154.         mask = x[:, mi:]
155.
156.         # 如果允许多标签，则为每个类别分配一个标签，否则只保留最佳类别
157.         if multi_label:
158.             i, j = (x[:, 5:mi] > conf_thres).nonzero(as_tuple=False).
                T
159.             x = torch.cat((box[i], x[i, 5 + j, None], j[:, None].float(
                t(), mask[i]), 1)
160.         else:
161.             conf, j = x[:, 5:mi].max(1, keepdim=True)
162.             x = torch.cat((box, conf, j.float(), mask), 1)[conf.view(
                -1) > conf_thres]
163.
164.         # 如果指定了要检测的类别，则过滤其他类别的检测框
165.         if classes is not None:
166.             x = x[(x[:, 5:6] == torch.tensor(classes, device=x.device))
                .any(1)]
167.

```

```

168.     # 检查检测框数量
169.     n = x.shape[0]
170.     if not n: # 如果没有检测框，则跳过
171.         continue
172.     elif n > max_nms: # 如果检测框数量超过限制，则按置信度排序后取前
        max_nms 个
173.         x = x[x[:, 4].argsort(descending=True)[:max_nms]]
174.     else:
175.         # 如果检测框数量没有超过限制，则按置信度排序
176.         x = x[x[:, 4].argsort(descending=True)]
177.
178.     # 执行批量 NMS
179.     c = x[:, 5:6] * (0 if agnostic else max_wh) # 类别偏移
180.     boxes, scores = x[:, :4] + c, x[:, 4] # 检测框 (偏移类别), 分数
181.     # 执行 NMS, 返回保留的检测框索引
182.     i = torchvision.ops.nms(boxes, scores, iou_thres)
183.
184.     # 如果检测到的物体数量超过最大检测数量，则只保留前 max_det 个
185.     if i.shape[0] > max_det:
186.         i = i[:max_det]
187.
188.     # 将当前图像的检测结果添加到输出列表中
189.     output[xi] = x[i]
190.     # 如果在使用 Apple MPS, 则将结果移回原始设备
191.     if mps:
192.         output[xi] = output[xi].to(device)
193.
194.     # 如果 NMS 执行时间超过时间限制，则打印警告并退出循环
195.     if (time.time() - t) > time_limit:

```

```

196.         print(f'WARNING ⚠ NMS time limit {time_limit:.3f}s exceed
           ed')
197.         break
198.
199. # 返回所有图像的检测结果
200. return output
201.
202. # 定义 xywh2xyxy 函数，用于将 nx4 边界框从 [x, y, w, h] 格式转换为
    [x1, y1, x2, y2] 格式
203. # 其中 xy1 表示左上角坐标，xy2 表示右下角坐标
204. def xywh2xyxy(x):
205.     # 如果输入是 torch.Tensor 类型，则克隆数据；如果是 numpy 数组，则
        复制数据
206.     y = x.clone() if isinstance(x, torch.Tensor) else np.copy(x)
207.     # 计算左上角 x 坐标
208.     y[:, 0] = x[:, 0] - x[:, 2] / 2
209.     # 计算左上角 y 坐标
210.     y[:, 1] = x[:, 1] - x[:, 3] / 2
211.     # 计算右下角 x 坐标
212.     y[:, 2] = x[:, 0] + x[:, 2] / 2
213.     # 计算右下角 y 坐标
214.     y[:, 3] = x[:, 1] + x[:, 3] / 2
215.     return y
216.
217. # 定义 get_labels_from_txt 函数，用于从文本文件中读取标签并创建字典
218. def get_labels_from_txt(path):
219.     labels_dict = dict() # 初始化一个空字典来存储标签
220.     with open(path) as f: # 打开文件
221.         for cat_id, label in enumerate(f.readlines()): # 遍历文件
            中的每一行

```

```

222.         labels_dict[cat_id] = label.strip() # 将读取的标签去除
           两端空白后存入字典
223.     return labels_dict # 返回标签字典
224.
225. # 定义 scale_coords 函数, 用于调整坐标点以适应不同尺寸的图像
226. def scale_coords(img1_shape, coords, img0_shape, ratio_pad=None):
227.     # 根据 img1_shape 和 img0_shape 调整坐标
228.     if ratio_pad is None: # 如果没有提供 ratio_pad, 则从
           img0_shape 计算
229.         gain = min(img1_shape[0] / img0_shape[0], img1_shape[1] /
           img0_shape[1]) # 计算缩放比例
230.         pad = (img1_shape[1] -
           img0_shape[1] * gain) / 2, (img1_shape[0] -
           img0_shape[0] * gain) / 2 # 计算填充量
231.     else: # 如果提供了 ratio_pad, 则使用提供的值
232.         gain = ratio_pad[0][0]
233.         pad = ratio_pad[1]
234.
235.     # 应用 x 方向的填充
236.     coords[:, [0, 2]] -= pad[0]
237.     # 应用 y 方向的填充
238.     coords[:, [1, 3]] -= pad[1]
239.     # 应用缩放比例
240.     coords[:, :4] /= gain
241.     # 裁剪坐标点, 确保它们在有效范围内
242.     clip_coords(coords, img0_shape)
243.     return coords # 返回调整后的坐标点
244.
245. # 定义 clip_coords 函数, 用于裁剪边界框坐标, 确保它们在图像尺寸范围内
246. def clip_coords(bboxes, shape):

```

Error! Unknown document property name.

```

247.     # 如果 boxes 是 torch.Tensor 类型, 则逐个快速裁剪
248.     if isinstance(boxes, torch.Tensor):
249.         boxes[:, 0].clamp_(0, shape[1]) # 裁剪 x1 坐标, 确保在 0 到
           图像宽度之间
250.         boxes[:, 1].clamp_(0, shape[0]) # 裁剪 y1 坐标, 确保在 0 到
           图像高度之间
251.         boxes[:, 2].clamp_(0, shape[1]) # 裁剪 x2 坐标, 确保在 0 到
           图像宽度之间
252.         boxes[:, 3].clamp_(0, shape[0]) # 裁剪 y2 坐标, 确保在 0 到
           图像高度之间
253.     # 如果 boxes 是 numpy 数组, 则整体快速裁剪
254.     else:
255.         boxes[:, [0, 2]] = boxes[:, [0, 2]].clip(0, shape[1]) #
           裁剪 x1 和 x2
256.         boxes[:, [1, 3]] = boxes[:, [1, 3]].clip(0, shape[0]) #
           裁剪 y1 和 y2
257.
258. # 定义 nms 函数, 作为非极大值抑制 (NMS) 的封装, 调用
           non_max_suppression 函数
259. def nms(box_out, conf_thres=0.4, iou_thres=0.5):
260.     try:
261.         # 尝试使用 multi_label=True 的 NMS
262.         boxout = non_max_suppression(box_out, conf_thres=conf_thr
           es, iou_thres=iou_thres, multi_label=True)
263.     except:
264.         # 如果出错, 则使用默认的 NMS
265.         boxout = non_max_suppression(box_out, conf_thres=conf_thr
           es, iou_thres=iou_thres)
266.     return boxout # 返回 NMS 处理后的结果
267.

```



```

268. # 定义 draw_bbox 函数, 在图像上绘制边界框和标签
269. def draw_bbox(bbox, img0, color, wt, names):
270.     det_result_str = '' # 初始化检测结果字符串
271.     for idx, class_id in enumerate(bbox[:, 5]): # 遍历边界框和类别
        ID
272.         if float(bbox[idx][4]) < float(0.05): # 如果置信度小于
            0.05, 则跳过
273.             continue
274.         print(str(idx) + ' ' + names[int(class_id)]) # 打印边界框
            索引和类别名称
275.         # 在图像上绘制矩形边界框
276.         img0 = cv2.rectangle(img0, (int(bbox[idx][0]), int(bbox[i
            dx][1])), (int(bbox[idx][2]), int(bbox[idx][3])), color, wt)
277.         # 在边界框左上角绘制类别名称
278.         img0 = cv2.putText(img0, str(idx) + ' ' + names[int(class
            _id)], (int(bbox[idx][0]), int(bbox[idx][1] + 16)), cv2.FONT_HERSHEY_
            _SIMPLEX, 0.5, (0, 0, 255), 1)
279.         # 在边界框下方绘制置信度
280.         img0 = cv2.putText(img0, '{:.2f}'.format(bbox[idx][4]), (
            int(bbox[idx][0]), int(bbox[idx][1] + 32)), cv2.FONT_HERSHEY_SIMPLEX,
            0.5, (0, 0, 255), 1)
281.         # 将检测结果添加到字符串
282.         det_result_str += '{} {} {} {} {} {} \n'.format(names[bbox
            [idx][5]], str(bbox[idx][4]), bbox[idx][0], bbox[idx][1], bbox[idx][
            2], bbox[idx][3])
283.     return img0 # 返回绘制了边界框和标签的图像

```

letterbox: 这个函数用于将图像调整到指定的新尺寸, 同时保持长宽比, 通过添加边框来填充图像。它还计算了缩放比例和填充尺寸。

non_max_suppression: 这个函数实现了非极大值抑制 (NMS), 用于从预测

结果中去除重叠的检测框。它接受多个参数，包括置信度阈值、IoU 阈值等，并返回经过 NMS 处理的检测结果。

`xywh2xyxy`: 将坐标格式从(x, y, w, h)转换为(x1, y1, x2, y2)，即从中心点和宽高转换为左上角和右下角的坐标。

`get_labels_from_txt`: 从文本文件中读取类别标签，并将它们存储在字典中。

`scale_coords`: 将坐标从一个图像尺寸调整到另一个图像尺寸，考虑了缩放和平移。

`clip_coords`: 将坐标限制在给定图像尺寸的范围内，确保检测框不会超出图像边界。

`nms`: 一个包装函数，用于调用 `non_max_suppression` 函数并返回 NMS 处理后的检测框。

`draw_bbox`: 在图像上绘制检测框，并在框旁边显示类别名称和置信度。

6.6. 封装线程

`ThreadTemplates.py` 定义了一个名为 `MyRepeatThread` 的自定义线程类，用于执行周期性动作，并在达到时间阈值或特定条件时停止。它包含了日志记录、属性打印、时间检查和停止方法。代码还定义了三个动作函数（前进、平移、旋转），每个函数创建一个 `MyRepeatThread` 线程实例以执行对应的机器狗动作。这些动作函数被组织在 `actions_dict` 字典中，便于根据动作 ID 快速索引。此外，还有一个 `action_the_pitch_angle` 函数用于调整机器狗的俯仰角。所有线程实例在创建时都不会立即启动，而是需要外部显式调用 `start()` 方法。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
```

```

4. import threading
5. import time
6. import logging
7.
8. # 导入与机器狗运动控制、指令发送和网络通信相关的模块
9. from speeds.sportspeed import *
10. from sendcommand import SendToCommand, heartbeat
11. from socketnetwork import network_utils
12.
13. # 配置日志记录, 设置日志级别为 DEBUG
14. logging.basicConfig(level=logging.DEBUG)
15.
16. # 使用 network_utils 中的 setup_socket_and_address 函数初始化网络通
    信, 设置套接字和目标地址
17. sfd, target_address = network_utils.setup_socket_and_address()
18.
19. class MyRepeatThread(threading.Thread):
20.     # MyRepeatThread 类的构造函数, 接受线程名称、动作函数、时间间隔和其
        他参数
21.     def __init__(self, name, action, interval, time_limit = None,
        *args) -> None:
22.         super(MyRepeatThread, self).__init__() # 调用父类构造函数
23.         self.name = name # 线程名称
24.         self.action = action # 线程要执行的动作函数
25.         self.interval = interval # 执行动作的时间间隔
26.         self.time_limit = time_limit # 线程执行的最大时间阈值
27.         self.args = args # 传递给动作函数的参数
28.         self.stopped = threading.Event() # 创建一个线程停止事件
29.         self.start_time = time.monotonic() # 记录线程开始执行的时
        间

```

```

30.         self.global_var = 0 # 用于存储全局变量
31.         self.current_time = time.monotonic() # 记录当前时间
32.         self.current_time_start = 0 # 记录动作开始的时间
33.
34.         # 线程的 run 方法，定义了线程要执行的代码
35.         def run(self) -> None:
36.             logging.info(f"Starting {self.name}") # 记录线程开始的信息
37.             try:
38.                 while not self.stopped.is_set(): # 如果线程停止事件未
                    被设置，则继续循环
39.                     self.current_time = time.monotonic() # 更新当前时
                        间
40.
41.                 # 如果设置了时间阈值并且当前时间超过时间阈值，则停止线
                    程
42.                 if self.time_limit is not None and self.check_tim
                    e_and_stop(self.current_time):
43.                     break
44.
45.             try:
46.                 self.action(*self.args) # 执行动作函数，并将参
                    数展开
47.             except KeyboardInterrupt:
48.                 self.stopped = True # 如 果 遇 到
                    KeyboardInterrupt 异常，则设置停止事件
49.             finally:
50.                 pass # finally 块通常用于执行无论 try 块是否成
                    功的操作，这里没有具体操作
51.

```

```

52.          # 计算自动作开始以来经过的时间，并计算需要休眠多长时间
           以保持固定频率
53.          action_start_time = time.monotonic()
54.          elapsed_time = action_start_time -
           self.current_time
55.          time_to_wait = max(0, self.interval -
           elapsed_time)
56.          time.sleep(time_to_wait) # 休眠以保持固定频率
57.
58.      finally:
59.          self.stopped.set() # 确保线程停止事件被设置
60.          logging.info(f"Exiting thread: {self.name}") # 记录线
           程退出的信息
61.
62. # 定义检查是否超过时间阈值并停止线程的方法
63. def check_time_and_stop(self, current_time) -> bool:
64.     # 如果当前时间与线程开始时间的差值超过时间阈值，则记录日志信息，设置
           停止事件，并返回 True
65.     if current_time - self.start_time > self.time_limit:
66.         logging.info(f'{self.name}由于超过时间阈值{self.time_limit}
           秒，系统自动停止! ')
67.         self.stopped.set()
68.         return True
69.     # 如果全局变量 global_var 等于 1，则设置停止事件，并返回 True
70.     elif self.global_var == 1:
71.         self.stopped.set()
72.         return True
73.     # 如果以上条件都不满足，则返回 False
74.     return False
75.

```

```

76. # 定义停止线程的方法
77. def stop(self) -> None:
78.     # 设置停止事件，停止线程
79.     self.stopped.set()
80.
81. # 定义打印对象所有属性及其值的方法
82. def print_attributes(self) -> dict:
83.     # 更新当前时间开始时间
84.     self.current_time_start = time.monotonic() - self.start_time
85.     # 返回一个字典，包含对象的所有属性和值
86.     return {attr: value for attr, value in vars(self).items()}
87.
88. # 定义前进动作的函数，返回一个线程对象
89. def action_go_straight(long, speedgear=3) -> MyRepeatThread:
90.     # 调用 go_straight 函数计算前进的距离和速度参数
91.     times, val = go_straight(long, speedgear)
92.     # 创建并返回一个线程对象，用于执行前进动作
93.     ACTION_pan_back_and_forth_thread = MyRepeatThread("ACTION_pan
    _back_and_forth_thread", SendToCommand.perform_action,
94.                                                         0.1, times,
    sfd, target_address, 0x21010130, val, 0)
95.     # 这里没有启动线程，start() 被注释掉了
96.     return ACTION_pan_back_and_forth_thread
97.
98. # 定义平移动作的函数，返回一个线程对象
99. def action_turn_left_and_right(long, speedgear=3) -> MyRepeatThre
    ad:
100.    # 调用 translate_left_and_right 函数计算平移的距离和速度参数
101.    times, val = translate_left_and_right(long, speedgear)
102.    # 创建并返回一个线程对象，用于执行平移动作

```

```

103.     ACTION_turn_left_and_right_thread = MyRepeatThread("ACTION_turn_left_and_right_thread", SendToCommand.perform_action,
104.                                     0.1, times, sfd, target_address, 0x21010131, val, 0)
105.     # 这里没有启动线程, start() 被注释掉了
106.     return ACTION_turn_left_and_right_thread
107.
108. # 定义旋转动作的函数, 返回一个线程对象
109. def action_revolve_left_and_right(angle) -> MyRepeatThread:
110.     # 调用 revolve_left_and_right 函数计算旋转的角度和速度参数
111.     times, val = revolve_left_and_right(angle)
112.     # 创建并返回一个线程对象, 用于执行旋转动作
113.     ACTION_revolve_left_and_right = MyRepeatThread("ACTION_revolve_left_and_right", SendToCommand.perform_action,
114.                                     0.1, times, sfd, target_address, 0x21010135, val, 0)
115.     # 这里没有启动线程, start() 被注释掉了
116.     return ACTION_revolve_left_and_right
117.
118. # 定义一个字典, 用于根据动作 ID 快速获取对应的动作函数
119. actions_dict = {
120.     1: action_go_straight,
121.     2: action_turn_left_and_right,
122.     3: action_revolve_left_and_right,
123.     # 其他函数可以根据实际情况添加到字典中
124. }
125.
126. # 定义调整俯仰角的动作函数, 返回一个线程对象
127. def action_the_pitch_angle():
128.     # 创建并返回一个线程对象, 用于执行调整俯仰角的动作

```

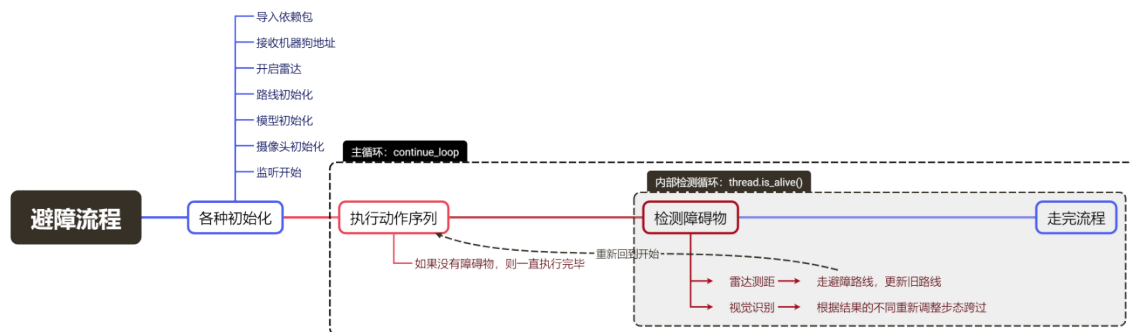
```

129.     ACTION_Adjust_the_pitch_angle = MyRepeatThread("ACTION_Adjust
        _the_pitch_angle", SendToCommand.perform_action,
130.                                     0.1, 20, sfd, target_address, 0x21010
        130, 15000, 0)
131.     # 这里没有启动线程, start() 被注释掉了
132.     return ACTION_Adjust_the_pitch_angle

```

6.7. 避障主程序

obstacle_main.py 是一个用于控制机器狗的自动化脚本，它集成了网络通信、多线程控制、避障、楼梯和坑洞识别，以及状态管理。脚本通过发送指令控制机器狗，实现自主导航，并能根据环境反馈调整行动路线。主要功能包括初始化通信、启动监听和处理线程、执行避障动作、处理特殊地形（楼梯和狗洞），以及根据机器狗状态发送相应指令。



6.7.1. 导入依赖包

它导入了多个模块以支持线程管理、通信、命令发送和状态监听。脚本设置了日志记录、全局变量和锁，定义了避障距离限制和行动路线。脚本还包括了避障动作参数和序列，以及用于控制线程暂停和继续的事件对象。整体上，这段代码构成了机器狗自动导航和避障功能的基础架构。

```

1.
2. #!/usr/bin/env python
3. # -*- coding: utf-8 -*-
4.
5.
6. import threading

```



```
7. import time
8. import logging
9. import math
10. from contextlib import ExitStack
11.
12. # 导入定义的机器狗相关的包
13. from threading_utils.ThreadTemplates import * # 线程的模板与基本轴
    指令的发送
14. from sendcommand import SendToCommand, heartbeat # 发送简单指令与
    心跳包
15. from socketnetwork import network_utils # 通信函数
16. from command.udp_command import * # 存放各种结构体与状态数据、指令
17. from speeds.sportspeed import * # 运动精准控制的基础版模型
18. from robotstatuswatcher.listener import * # 监听机器狗状态
19. from HandDistance.ridge_np import * # 测量距离
20. from obstacle_model_cap import *
21.
22. # 初始化日志，级别为最低级
23. logging.basicConfig(level=logging.DEBUG)
24.
25. # 全局变量和锁
26. status_list = []
27. status_list_lock = threading.Lock()
28. result = []
29. result_lock = threading.Lock()
30.
31. # 避障的距离限制
32. obs_void_distance = 0.4
33.
34. # 固定路线
```

```

35. actions_params = {
36.     1: [(2,), (1.1,)],
37.     3: [(90,)]
38. }
39. actions_sequence = [(1, 0), (3, 0), (1, 1)]
40.
41. # 避障路线
42. avoid_actions_params = {
43.     1: [(0.7,), (0.7,)],
44.     3: [(-46.0,), (91.0,), (-30.0,)]
45. }
46. avoid_actions_sequence = [(3, 0), (1, 0), (3, 1), (1, 1), (3, 2)]
47.
48. # 更新后的路线
49. updated_actions_params = actions_params.copy()
50.
51. # 避障次数
52. obstacle = 0
53. staircase = 0
54. hole = 0
55.
56. # 记录队列的角标
57. sequence_index = 0
58.
59. # 循环控制变量
60. continue_loop = True
61.
62. # 创建一个全局的 Event 对象用于控制线程暂停和继续
63. pause_event = threading.Event()

```

Error! Unknown document property name.

64. `pause_event.set()` # 初始状态为设置，表示不暂停

6.7.2. 获取机器狗接收地址

这段代码定义了一个名为 `initialize_communication` 的函数，其主要作用是初始化网络通信。函数通过调用 `network_utils.setup_socket_and_address` 方法设置套接字和目标地址，然后创建并启动一个心跳包发送线程，以保持与机器狗的持续通信。最后，函数返回初始化的套接字和目标地址。

```
1. def initialize_communication():
2.     # 设置网络通信，初始化套接字和目标地址
3.     sfd, target_address = network_utils.setup_socket_and_address()
4.     # 创建并启动心跳包发送线程
5.     heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)
6.     heartbeat_thread.start()
7.     # 返回套接字和目标地址
8.     return sfd, target_address
```

6.7.3. 发送指令以开启雷达

这段代码定义了一个名为 `initialize_threads` 的函数，负责初始化并启动多个线程。函数首先发送指令以开启机器狗的雷达功能，然后创建并启动两个线程：一个是雷达状态监听线程，用于持续监听雷达状态；另一个是图像识别处理线程，用于执行图像推理任务。这些线程的创建和启动为机器狗的状态监控和环境感知提供了并行处理能力。

```
1. def initialize_threads():
2.     # 向机器狗发送指令以开启雷达
3.     SendToCommand.perform_action(sfd, target_address, 0x21012109, 0x40, 0)
4.     # 创建雷达状态监听线程并启动
5.     radar_thread = threading.Thread(target=status_listener_radar, args=(status_list, status_list_lock))
```

```

6.     radar_thread.start()
7.     # 创建图像识别处理线程并启动
8.     picture_thread = threading.Thread(target=inference_loop, args=(r
    esult, result_lock))
9.     picture_thread.start()

```

6.7.4. 遍历避障动作

这段代码定义了一个名为 `execute_avoid_sequence` 的函数，其作用是执行避障动作序列。函数通过遍历避障动作参数和序列，获取每个动作的函数和参数。如果参数是元组类型，函数将创建并启动一个新线程来执行相应的避障动作，并等待该线程完成。此外，函数在执行前后打印日志信息以标明避障动作的开始和结束。整个过程用于在遇到障碍物时控制机器狗进行避障。

```

1. def execute_avoid_sequence(params=avoid_actions_params, sequence=avo
    id_actions_sequence):
2.     logging.info("进入 execute_avoid_sequence")
3.     logging.info("-----params-----: %s", params)
4.     # 遍历避障动作序列
5.     for action_id, params_index in sequence:
6.         # 根据动作 ID 获取对应的动作函数
7.         action_func = actions_dict.get(action_id, None)
8.         if action_func:
9.             # 获取避障参数
10.            avoid_params = params[action_id][params_index]
11.            # 如果参数是元组类型，则创建并启动避障动作线程
12.            if isinstance(avoid_params, tuple):
13.                avoid_thread=action_func(*avoid_params)
14.                avoid_thread.start()
15.                avoid_thread.join()
16.            time.sleep(1)

```

```
17. logging.info("离开 execute_avoid_sequence")
```

6.7.5. 更新行走路线参数

这段代码定义了一个名为 `update_route_params` 的函数，它用于更新机器狗的行走路线参数。函数接收当前线程、避障前的行走距离、临时变量作为参数，并从全局变量 `updated_actions_params` 中获取和更新当前动作序列的参数。新的行走距离通过从原始参数中减去已行走的距离和临时变量来计算。最后，函数记录并打印更新后的路线参数。这个过程是为了在避障后重新计算并设置机器狗的下一步行动路线。

```
1. def update_route_params(thread, params, before_long, temp):
2.     # global updated_actions_params
3.     # 更新机器狗的行走路线参数
4.     # 这里 updated_actions_params 是全局变量，存储更新后的路线参数
5.     # 获取当前动作序列的索引和键
6.     current_params_index = actions_sequence[sequence_index]
7.     current_params_key, current_params_index = actions_sequence[sequence_index][0], actions_sequence[sequence_index][1]
8.     # 更新路线参数，计算新的行走距离
9.     updated_actions_params[current_params_key][current_params_index]
        = (params[0] - before_long - temp,)
10.    logging.info(f'更新后的路线
        updated_actions_params: {updated_actions_params}')
```

6.7.6. 定义全局变量和局部变量

这段代码定义了一系列函数，用于控制机器狗的行为和响应不同环境情况：

- (1) `initialize_communication` 初始化网络通信，设置套接字和目标地址，并启动心跳包发送线程。
- (2) `initialize_threads` 初始化多个线程，包括雷达监听和图像识别处理。

- (3) `main_loop` 是主循环，遍历执行预定义的动作序列，处理避障、楼梯和坑洞，并根据状态更新行动路线。
- (4) `handle_obstacles` 处理避障逻辑，更新路线参数，并在遇到障碍时执行避障动作序列。
- (5) `handle_staircases` 和 `handle_holes` 分别处理楼梯和坑洞的检测，发送指令以调整机器狗的行为。
- (6) `handle_status` 根据机器狗的状态发送相应的控制指令。
- (7) 脚本最后检查 `__name__` 来确定是否作为主程序运行，并调用上述函数来启动整个过程。

整体上，这段代码实现了一个自动化的控制流程，使机器狗能够根据环境反馈自主导航和响应障碍。

```
1. def main_loop(sfd, target_address):
2.     # 定义全局变量和局部变量
3.     global continue_loop, sequence_index, actions_params, actions_
       sequence
4.     while continue_loop:
5.         continue_loop = False # 重置循环控制变量
6.         # 遍历动作序列
7.         for action_id, params_index in actions_sequence:
8.             # 根据动作ID 获取对应的动作函数
9.             action_func = actions_dict.get(action_id, None)
10.            if action_func:
11.                # 获取动作参数
12.                params = actions_params[action_id][params_index]
13.                # 打印当前参数
14.                logging.info(f'params: {params}')
```

```

15.         # 如果参数是元组类型，则创建线程执行动作
16.         if isinstance(params, tuple):
17.             thread = action_func(*params)
18.             thread.start()    # 启动线程
19.             # 等待线程执行完毕
20.             while thread.is_alive():
21.                 # 使用上下文管理器确保线程安全地访问状态列表和
                结果列表
22.                 with ExitStack() as stack:
23.                     stack.enter_context(status_list_lock)
24.                     stack.enter_context(result_lock)
25.                     # 处理障碍、楼梯、坑洞和状态
26.                     handle_obstacles(thread, params)
27.                     handle_staircases(sfd, target_address)
28.                     handle_holes(sfd, target_address)
29.                 # 如果循环控制变量被设置为False，则退出循环
30.                 if not continue_loop:
31.                     # 增加队列角标
32.                     sequence_index += 1 # 成功走完一个，队列角
                标加一
33.                 thread.join()
34.                 handle_status(sfd, target_address)
35.                 # 如果循环控制变量被设置为True，则跳出当前循环
36.                 if continue_loop:
37.                     break
38.                 time.sleep(1)
39.
40.
41. def handle_obstacles(thread, params):

```

```

42.     # 定义全局变量
43.     global continue_loop, obstacle
44.     # 如果未遇到障碍物，并且状态列表中有数据，并且距离小于等于避障距离限制
45.     if obstacle == 0 and status_list and status_list[3] <= obs_void_distance:
46.         # 增加障碍物计数
47.         obstacle += 1
48.         # 执行直线前进动作，参数为长距离，当前时间，避障距离限制
49.         before_long = go_straight(long=9999, times=thread.print_at_tributes()['current_time_start'], obs_void_distance=obs_void_distance)
50.         # 计算临时变量
51.         temp = 2 * avoid_actions_params[1][0][0] / math.sqrt(2)
52.         # 更新路线参数
53.         update_route_params(thread, params, before_long, temp)
54.         # 停止当前线程
55.         thread.stop()
56.         thread.join()
57.         # 打印停止信息
58.         logging.info('原始路线停止。')
59.         # 如果线程已停止
60.         if thread._is_stopped:
61.             # 执行避障序列
62.             execute_avoid_sequence()
63.             # 重置动作参数和序列
64.             global actions_params, actions_sequence
65.             actions_params = updated_actions_params
66.             actions_sequence = actions_sequence[sequence_index:]
67.             # 设置循环控制变量为True，继续执行

```



```

68.         continue_loop = True
69.         # 清除暂停事件
70.         pause_event.clear()
71.
72.
73. def handle_staircases(sfd, target_address):
74.     # 定义全局变量
75.     global staircase
76.     # 如果未遇到楼梯, 并且结果列表为['vegetables'], 表示检测到楼梯
77.     if staircase == 0 and result == ['vegetables']:
78.         # 增加楼梯计数
79.         staircase += 1
80.         # 打印检测到楼梯的信息
81.         logging.info("Detected 'vegetables', pausing thread.")
82.         # 发送指令以暂停机械狗
83.         SendToCommand.perform_action(sfd, target_address, 0x210104
            07, 0, 0)
84. def handle_holes(sfd, target_address):
85.     # 定义全局变量
86.     global hole
87.     # 如果未遇到坑洞, 并且结果列表为['drink'], 表示检测到坑洞
88.     if hole == 0 and result == ['drink']:
89.         # 增加坑洞计数
90.         hole += 1
91.         # 打印检测到坑洞的信息
92.         logging.info("Detected 'drink', pausing thread.")
93.         # 发送指令以暂停机械狗
94.         SendToCommand.perform_action(sfd, target_address, 0x210104
            06, 0, 0)

```

```

95. def handle_status(sfd, target_address):
96.     # 定义全局变量
97.     global hole
98.     # print(status_listener())
99.     if status_listener() == [6, 13, 1]: # 力控状态（静止站立）且步
        态为高踏步越障步态
100.         SendToCommand.perform_action(sfd, target_address, 0x21
            010300, 0, 0)
101.         time.sleep(1)
102.         elif status_listener() == [6, 0, 1] and hole == 1: # 正在
            以平地低速步态踏步（匍匐状态）
103.             # 增加坑洞计数
104.             hole += 1
105.             SendToCommand.perform_action(sfd, target_address, 0x21
                010406, 0, 0)
106.             time.sleep(1)
107. if __name__ == "__main__":
108.     # 建立通讯地址
109.     sfd, target_address = initialize_communication()
110.     # 初始化模型、雷达、识别
111.     initialize_threads()
112.     # 站立
113.     SendToCommand.perform_action(sfd, target_address, 0x210102
        02, 0, 0)
114.     # 2 秒延时保证机器狗站稳再开始执行任务
115.     time.sleep(2)
116.     # 进入主程序
117.     main_loop(sfd, target_address)

```

6.8. 识别和获取避障信息

obstacle_model_cap.py 实现了一个基于深度学习的图像检测系统。它通过 getImage 获取图像，使用 letterbox 进行尺寸调整，然后通过 Image_inference 函数进行模型推理，最后通过 inference_loop 函数连续检测图像中的对象。系统能够处理多线程并发，并在检测结果稳定时更新结果列表。支持通过键盘中断退出。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. import cv2 # 导入 OpenCV 库，用于图像处理
5. import numpy as np # 导入 NumPy 库，用于对多维数组进行计算
6. import torch # 导入 PyTorch 库，用于深度学习运算
7. from mindx.sdk import Tensor # 从 MindX SDK 导入 Tensor 数据结构
8. from mindx.sdk import base # 从 MindX SDK 导入基础推理接口
9. import time # 导入 time 模块，用于时间相关的操作
10. import logging # 导入 logging 库，用于日志记录
11. import threading # 导入 threading 库，用于多线程操作
12. from typing import * # 导入 typing 模块，用于类型注解
13. import sys # 导入 sys 库，用于系统相关操作
14. sys.path.append('camera/') # 将'camera'目录添加到模块搜索路径
15. from HKcamera import getImage # 从 HKcamera 模块导入 getImage 函数，
    用于获取图像
16. from det_utils import get_labels_from_txt, letterbox, scale_coord
    s, nms, draw_bbox # 从 det_utils 模块导入相关函数
17.
18. # 初始化日志，设置日志级别为DEBUG
19. logging.basicConfig(level=logging.DEBUG)
20.
21. # 定义 Image_inference 函数，用于执行图像推理
22. def Image_inference(model, labels_dict):
```

```

23.         try:
24.             img = getImage() # 获取图像
25.             frame = img.copy() # 复制图像用于后续处理
26.             # 使用 letterbox 函数对图像进行尺寸调整和填充
27.             img, scale_ratio, pad_size = letterbox(frame, new_shape=[
                640, 640])
28.             ill_sets = [] # 初始化用于存储检测结果的列表
29.             # 对图像进行格式转换和归一化处理, 准备输入到模型中
30.             img = img[:, :, ::-1].transpose(2, 0, 1)
31.             img = np.expand_dims(img, 0).astype(np.float32)
32.             img = np.ascontiguousarray(img) / 255.0
33.             img = Tensor(img)
34.             # 使用模型进行推理
35.             output = model.infer([img])[0]
36.             output.to_host() # 将推理结果从设备传输到主机
37.             output = np.array(output) # 将推理结果转换为 NumPy 数组
38.             # 使用 nms 函数进行非极大值抑制处理
39.             boxout = nms(torch.tensor(output), conf_thres=0.7, iou_th
                res=0.5)
40.             # 将 nms 处理后的预测结果转换为 NumPy 数组
41.             pred_all = boxout[0].numpy()
42.             # 调整坐标点以适应原始图像尺寸
43.             scale_coords([640, 640], pred_all[:, :4], frame.shape, ra
                tio_pad=(scale_ratio, pad_size))
44.             # 遍历预测结果, 提取类别和置信度信息
45.             for idx, class_id in enumerate(pred_all[:, 5]):
46.                 # 记录检测到的类别、类别 ID 和置信度
47.                 ill_sets.append([labels_dict[int(class_id)], int(clas
                    s_id), round(pred_all[idx][4], 2)])
48.             # 返回检测结果列表

```

```

49.         return ill_sets
50.     except KeyboardInterrupt:
51.         # 如果发生键盘中断 (Ctrl+C), 返回空列表
52.         return []
53.
54. # 定义 model_init 函数, 用于初始化模型和标签字典
55. def model_init(model_path, device_id, label_path):
56.     base.mx_init() # 初始化 MindX SDK
57.     model = base.model(modelPath=model_path, deviceId=device_id)
58.     # 加载模型
59.     labels_dict = get_labels_from_txt(label_path) # 从标签文件中
60.     # 获取类别标签
61.     return model, labels_dict # 返回模型和标签字典
62.
63. def inference_loop(result, result_lock):
64.     # 初始化一个空列表, 用于存储推理结果
65.     list0 = []
66.     # 初始化计数器
67.     k = 0
68.
69.     # 设备 ID 设置为 0
70.     DEVICE_ID = 0
71.     # 模型路径设置为 'smart_logistics_models/1.om'
72.     model_path = 'smart_logistics_models/1.om'
73.     # 标签文件路径设置为
74.     'smart_logistics_models/predefined_classes.txt'
75.     label_path = 'smart_logistics_models/predefined_classes.txt'
76.     # 调用 model_init 函数初始化模型和标签字典
77.     model, labels_dict = model_init(model_path, DEVICE_ID, label_
    path)

```

```
75.
76.     try:
77.         # 进入一个无限循环，持续进行推理
78.         while True:
79.             # 调用 Image_inference 函数进行图像推理，并获取结果
80.             ill_sets = Image_inference(model, labels_dict)
81.             # 如果推理结果为空，继续下一次循环
82.             if not ill_sets:
83.                 continue
84.             # 将推理结果中的第一个元素添加到列表 list0 中
85.             list0.append(ill_sets[0][1])
86.             # 计数器 k 加 1
87.             k += 1
88.
89.             # 如果计数器 k 等于 2，表示已经收集到两个连续的推理结果
90.             if k == 2:
91.                 # 创建一个集合，用于存储 list0 中的唯一元素
92.                 unique_elements = set(list0)
93.                 # 如果集合的长度为 1，表示两次推理结果相同
94.                 if len(unique_elements) == 1:
95.                     # 将对象类别设置为 list0 中的第一个元素
96.                     object_class = ill_sets[0][0]
97.                     # 重置计数器 k 和 list0
98.                     k, list0 = 0, []
99.                     # 使用结果锁进行线程安全操作
100.                    with result_lock:
101.                        # 清空结果列表
102.                        result.clear()
103.                        # 将新的结果添加到结果列表中
104.                        result.append(object_class)
```

Error! Unknown document property name.

```

105.                                     # 打印当前结果
106.                                     print('result:', result)
107.                                     # 如果集合长度不为 1，表示两次推理结果不一致
108.                                     else:
109.                                     # 设置 object_class 为 None
110.                                     object_class = None
111.                                     # 重置计数器 k 和 list0
112.                                     k, list0 = 0, []
113.
114.     # 如果捕获到 KeyboardInterrupt 异常（例如使用 Ctrl+C），不执行任何
    操作
115.     except KeyboardInterrupt:
116.         pass

```

第七章 运行示例

首先在文件夹的目录下执行先执行这个指令

```

. /usr/local/Ascend/mxVision-
5.0.RC3/set_env.sh && . /usr/local/Ascend/ascend-toolkit/set_env.sh

```

然后再输入

```
python obstacle_main.py
```

让机器狗识别到相对应的类别即可。

```
(base) root@davinci-mini:/opt/UDPControl# python obstacle_main.py
成功连接到ip: 192.168.1.100, port: 43897
Begin to initialize Log.
The output directory of logs file exist.
WARNING: Logging before InitGoogleLogging() is written to STDERR
I20220408 03:48:30.542012 20473 FileUtils.cpp:331] The input file is empty
I20220408 03:48:30.542076 20473 FileUtils.cpp:475] Check Other group permission: Current permission is 4, but required no greater than 0.
Save logs information to specified directory.
Find 1 devices!

u3v device: [0]
device model name: MV-C5060-10UC-PRO
user serial number: DA0275184
/usr/local/miniconda3/lib/python3.9/site-packages/torchvision/io/image.py:13: UserWarning: Failed to load image Python extension:
warn(f"Failed to load image Python extension: {e}")
INFO:root:Starting HeartbeatThread
^Aresult: ['hole']
result: ['hole']
```